UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Distributed Systems and Web Technologies

LUCRARE DE DISERTAȚIE

Coordonator științific:
conf. dr. ing. Cristian Nicolae BUȚINCU

Absolvent:
George-Alexandru MIRON

Iași, 2026

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Distributed Systems and Web Technologies

Distributed Platform for Running Services Within a Cluster

LUCRARE DE DISERTAȚIE

Coordonator științific:
conf. dr. ing. Cristian Nicolae BUȚINCU

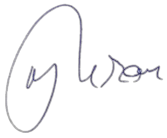
Absolvent:
George-Alexandru MIRON

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII LUCRĂRII DE DISERTAȚIE

Subsemnatul Miron George-Alexandru,
legitimat cu CI seria IS nr. 1003727 , CNP 1930524226711
autorul lucrării Distributed Platform for Running Services Within a Cluster

elaborată în vederea susținerii examenului de finalizare a studiilor de masterat, programul de studii Distributed Systems and Web Technologies organizat de către Facultatea de Automatică şi Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi" din Iaşi, sesiunea iulie a anului universitar 2025 - 2026, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi" din Iaşi (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) şi a convențiilor internaționale privind drepturile de autor.

Data	Semnătura
15.06.2026	

Cuprins

Distributed Platform for Running Services Within a Cluster

George-Alexandru MIRON

Abstract

Running services reliably across multiple machines requires solving several difficult problems, such as node discovery, scheduling, failure recovery and access control. Production platforms like Kubernetes solve these problems at the cost of high complexity. This thesis presents the design and implementation of a distributed platform, written from scratch in Go, that provides the essential cluster management mechanisms in a simple and understandable way.

The platform follows a master and worker architecture. Workers are discovered automatically through UDP multicast heartbeats, are grouped into resource pools and run Docker containers with enforced processor and memory limits. The cluster state is replicated across three master nodes using the Raft consensus protocol. Requests sent to a follower are forwarded transparently to the leader, so the failure of a master does not affect users.

The scheduler places the replicas of a service only when the cluster has enough capacity for all of them and chooses the nodes through a pluggable load balancing strategy. Failed nodes are detected within 30 seconds and their containers are restarted on healthy nodes. When a node that was considered dead returns, its duplicate containers are removed automatically. Access to the platform is protected by JWT authentication, role based access control and an audit log, all exposed through a REST API.

The platform was validated through unit tests and manual end to end scenarios covering node discovery, service deployment, fault recovery and security enforcement. The results confirm that the solution fits educational use and smaller deployments.

Keywords: distributed systems, cluster management, container orchestration, heartbeat, fault tolerance, load balancing, Raft consensus

Capitolul 1. Introduction

Running services reliably across multiple machines is a common challenge in modern software systems. As user traffic grows, a single server can no longer handle all requests. The application must be split across a cluster of machines, where each machine is called a node.

Managing a cluster brings several problems. The system needs to know which nodes are available. When a new service is deployed, it must decide which node should run it. If a node crashes, the services running on it must be moved to another node. The system also needs access control and a record of who did what and when.

Kubernetes is the most popular platform for solving these problems, but it has hundreds of components and requires a team of engineers to run. This thesis proposes a simpler alternative: a distributed platform built from scratch that implements the fundamental concepts (heartbeats, scheduling, load balancing, fault tolerance, security and audit) in a clear and understandable way.

1.1. Objectives

The main objective of this thesis is to design and implement a distributed platform for running containerized services within a cluster. The platform addresses the following specific objectives:

- **Automatic node discovery**. Worker nodes are discovered automatically by the master when they send their first heartbeat via UDP multicast. No explicit join or leave mechanism is needed.
- **Resource pools**. Nodes are grouped into pools based on their hardware resources (CPU, RAM). Services can be deployed to a specific pool, ensuring they run on appropriate hardware.
- **Load balancing**. When deploying a service, the platform selects the best node using a pluggable load balancing strategy (Least Connections or Weighted).
- **Fault tolerance**. The platform detects node failures through heartbeat monitoring and automatically restarts affected containers on healthy nodes. A Raft consensus protocol ensures master node availability.
- **Security**. User authentication is handled through JWT tokens and a role-based access control system (Admin, Writer, Reader) restricts what each user can do.
- **Audit**. All important actions are logged in an append-only audit log that can be queried through the API.

1.2. Methodology and Tools

The platform is implemented in Go, chosen for its built-in concurrency support (goroutines), efficient networking and suitability for systems programming. The main tools and libraries used are:

- **Go**. The programming language for both master and worker components
- **Docker SDK**. For managing containers on worker nodes
- **HashiCorp Raft**. For leader election and state replication between master nodes
- **UDP multicast**. For heartbeat communication between workers and master
- **JWT (JSON Web Tokens)**. For user authentication

1.3. Thesis Structure

The thesis is organized into five chapters:

- **Chapter 1. Introduction**: presents the context, motivation, objectives, methodology and an overview of the thesis structure.
- **Chapter 2. Theoretical Background and Similar Solutions**: covers the key concepts of

distributed systems (heartbeats, consensus, fault tolerance, load balancing, containerization) and compares existing platforms (Kubernetes, Docker Swarm and HashiCorp Nomad) with the proposed solution. It also defines the expected characteristics of the platform.

- **Chapter 3. Proposed Solution**: describes the architecture of the platform in detail, including the master-worker component design, domain model, heartbeat-based node discovery, fault tolerance mechanisms, load balancing strategies, REST API, service deployment flow, JWT authentication with refresh tokens, role-based access control, audit logging, Raft consensus for state replication and Docker integration.
- **Chapter 4. Testing and Experimental Results**: presents the testing methodology including automated unit tests for the load balancer, scheduler and access control components, as well as manual end-to-end test scenarios covering node discovery, service deployment, fault tolerance, security enforcement and audit logging.
- **Chapter 5. Conclusions**: summarizes the achieved objectives, compares the platform with existing solutions, discusses its limitations and proposes future improvements such as persistent storage, service networking and auto-scaling.

Capitolul 2. Theoretical Background and Similar Solutions

This chapter presents the key concepts behind distributed systems and reviews existing platforms that solve similar problems. Understanding these concepts is important for explaining the design decisions made in the proposed solution.

2.1. Distributed Systems

A distributed system is a collection of independent computers that appear to the user as a single system [1]. The main challenges in distributed systems are:

- **Partial failure**. Some nodes can fail while others continue to work. The system must handle this without stopping entirely.
- **No global clock**. Each node has its own clock and there is no single source of time for the entire system.
- **Network unreliability**. Messages between nodes can be delayed, lost or delivered out of order.

2.1.1. Node Discovery and Heartbeats

In a cluster, nodes need to know about each other. There are two common approaches:

Static configuration. The list of nodes is defined in a configuration file. Simple, but does not support dynamic scaling.

Heartbeat-based discovery. Each node periodically sends a small message (heartbeat) to announce that it is alive. If a node stops sending heartbeats, it is considered dead. This approach is used by systems like Consul [2] and the proposed platform.

Heartbeats can be sent via TCP (reliable, but more overhead) or UDP (lightweight, no connection setup). UDP multicast allows a single packet to reach multiple receivers at once, which is efficient when multiple master nodes need to receive heartbeats [3].

2.1.2. Consensus and Leader Election

When multiple master nodes need to agree on the state of the cluster, they use a consensus protocol. The most widely used consensus protocol in modern systems is **Raft** [4].

Raft works by electing a leader among the master nodes. The leader processes all write operations and replicates them to the followers. If the leader fails, the followers elect a new leader. Raft guarantees that:

- Only one leader exists at any time.
- All nodes agree on the same sequence of operations.
- The system continues to work as long as a majority of nodes (quorum) are alive. For 3 nodes, the quorum is 2.

2.1.3. Fault Tolerance

Fault tolerance is the ability of a system to continue operating when some of its components fail [1]. Common strategies include **replication** (running multiple copies of a service on different nodes so that if one copy fails, others continue to serve requests), **health monitoring** (periodically checking if nodes are alive via heartbeats and taking action when they are not) and **automatic recovery** (restarting failed services on healthy nodes without human intervention).

2.1.4. Load Balancing

Load balancing distributes work across multiple nodes to avoid overloading any single node. Common strategies include:

- **Round Robin**. Requests are distributed to nodes in a circular order.
- **Least Connections**. The node with the fewest active connections is chosen.
- **Weighted**. Nodes with more available resources receive more work.

The proposed platform implements Least Connections and Weighted strategies with a pluggable interface, allowing new strategies to be added without modifying existing code. In the proposed platform, the number of running containers on a node is used as its connection count.

2.1.5. Containerization

Containers are lightweight, isolated environments for running applications. Unlike virtual machines, containers share the host operating system kernel, which makes them faster to start and more efficient in resource usage [5].

Docker is the most widely used container runtime. It provides an API for creating, starting, stopping and removing containers. The proposed platform uses the Docker SDK to manage containers on worker nodes.

2.2. Similar Solutions

Several platforms exist for running containerized services in a cluster. This section compares the most relevant ones with the proposed solution.

2.2.1. Kubernetes

Kubernetes [6] is the industry standard for container orchestration. It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF).

Key features:

- Automatic scheduling and scaling of containers
- Self-healing: restarts failed containers, replaces nodes
- Service discovery and load balancing
- Configuration management and secrets
- Extensible through custom resources and operators

Kubernetes uses `etcd` (a distributed key-value store based on Raft) for cluster state and `kubelet` agents on each node for container management.

Comparison with the proposed platform: Kubernetes is a full production platform with hundreds of features. The proposed platform focuses on the core mechanisms (heartbeats, scheduling, fault tolerance, security) and implements them from scratch, providing a clear and simple architecture that is easier to understand and extend. Kubernetes is designed for large-scale production use, while the proposed platform is designed for educational purposes and smaller deployments.

2.2.2. Docker Swarm

Docker Swarm [7] is Docker's built-in orchestration tool. It is simpler than Kubernetes and integrates directly with the Docker CLI.

Key features:

- Uses the standard Docker API
- Built-in service discovery
- Load balancing via ingress routing

- Raft consensus for manager nodes
- TLS encryption for node communication

Comparison with the proposed platform: Docker Swarm is closer in simplicity to the proposed platform. Both use Raft for consensus and Docker for container management. The main differences are that the proposed platform uses UDP multicast for heartbeats (instead of TCP), does not require TLS for internal communication (trusted network assumption) and implements auto-join via heartbeats instead of explicit join tokens.

2.2.3. HashiCorp Nomad

Nomad [8] is a workload orchestrator developed by HashiCorp. Unlike Kubernetes, it can manage not only containers but also standalone applications, Java services and batch jobs.

Key features:

- Single binary, easy to deploy
- Multi-datacenter support
- Integrates with Consul for service discovery and Vault for secrets
- Uses Raft for consensus

Comparison with the proposed platform: Nomad shares the philosophy of simplicity. Both use Raft for consensus and a master-worker architecture. The proposed platform is more focused because it only manages Docker containers and implements all components (discovery, scheduling, security) in a single system, without external dependencies like Consul or Vault.

2.3. Summary of Comparison

Table 2.1 summarizes the key differences between the existing solutions and the proposed platform.

Tabelul 2.1. Comparison of existing solutions with the proposed platform

Feature	**Kubernetes**	**Swarm**	**Nomad**	**Proposed**
Consensus	etcd (Raft)	Raft	Raft	Raft
Node discovery	kubelet	join token	Consul	heartbeat
Heartbeat protocol	TCP	TCP	TCP	UDP multicast
Container runtime	multiple	Docker	multiple	Docker
Internal encryption	optional	TLS	TLS	none (trusted)
Complexity	very high	medium	medium	low
Load balancing	yes	yes	no (external)	yes
RBAC	yes	limited	yes	yes
Audit log	yes	no	yes	yes

2.4. Expected Characteristics

Based on the analysis of existing solutions and the identified gaps, the proposed platform should meet the following requirements:

- **Automatic node discovery**. Worker nodes should be detected automatically when they start, without manual configuration or explicit join commands.
- **Failure detection within 30 seconds**. The system should detect a dead node within 30 seconds of the last heartbeat and begin container recovery immediately.
- **Container recovery without data loss**. When a node dies, all service parameters (image, environment variables, ports, command) should be preserved in the cluster state, allowing containers to be restarted identically on healthy nodes.

- **Pluggable load balancing**. The load balancing strategy should be interchangeable without modifying the rest of the code.
- **Role-based access control**. The platform should support at least three roles (Admin, Writer, Reader) with clearly separated permissions.
- **Audit trail**. All important actions should be logged in an append-only log that can be queried through the API.
- **High availability**. The master should be replicated across at least 3 nodes using Raft consensus, tolerating the failure of 1 master node.
- **Low complexity**. The platform should be simple enough to understand deploy and extend, unlike production platforms such as Kubernetes.

2.5. Conclusions

The existing platforms are mature and feature-rich, but they come with significant complexity. Kubernetes, in particular, has a steep learning curve, requires extensive infrastructure (etcd cluster, multiple control plane components, networking plugins) and is designed for organizations with dedicated operations teams. Docker Swarm is simpler but has been largely superseded by Kubernetes in the industry. Nomad offers a middle ground but depends on external tools (Consul, Vault) for service discovery and secrets.

The proposed platform fills a gap by implementing the core distributed systems mechanisms in a single, self-contained system. It covers heartbeat-based node discovery using UDP multicast, automatic resource pool allocation, pluggable load balancing with two strategies, fault tolerance with container recovery and split-brain prevention, JWT-based authentication with refresh tokens, role-based access control with three permission levels and an append-only audit log. The cluster state is replicated across multiple master nodes using the Raft consensus protocol.

Unlike the existing solutions, the platform is designed with simplicity as a primary goal. The entire codebase is a single Go module with minimal external dependencies. This makes it suitable for educational use, where understanding the underlying concepts is more important than production readiness and for smaller deployments where Kubernetes would introduce unnecessary complexity.

Capitolul 3. Proposed Solution

The problem addressed by this thesis is running containerized services reliably across a cluster of machines. The key challenges are: discovering nodes automatically, distributing workloads efficiently, recovering from failures without downtime and controlling access to the platform. Existing solutions like Kubernetes solve these problems but are too complex for smaller deployments and educational use.

The proposed solution is a distributed platform built from scratch in Go that implements the core mechanisms needed to manage a cluster: heartbeat-based node discovery, resource pools, pluggable load balancing, automatic fault recovery, JWT-based security with role-based access control and an append-only audit log. The cluster state is replicated across multiple master nodes using the Raft consensus protocol.

This chapter describes the architecture in detail. The platform has two main components: the **master node**, which manages the cluster and the **worker nodes**, which run the containers.

3.1. Technology Choices

The platform is implemented in **Go**, chosen for its built-in concurrency model (goroutines and channels), efficient networking libraries, single-binary compilation and strong support for systems programming. Both the master and worker are written in Go.

Heartbeat communication uses **UDP multicast**, which is lightweight (no connection setup, no handshake) and allows a single packet to reach all master nodes simultaneously. This is more efficient than TCP for periodic status messages that do not require acknowledgment.

Docker is used as the container runtime on worker nodes. The Docker SDK for Go [9] provides a simple API for creating, starting and stopping containers programmatically.

For consensus and state replication between master nodes, the platform uses **HashiCorp Raft** [10], an open-source Go implementation of the Raft consensus protocol.

User authentication is handled through **JWT (JSON Web Tokens)**, a standard defined in RFC 7519 [11] and implemented in the platform with the golang-jwt library [12]. JWTs are stateless (no server-side session storage needed) and can encode the user's role directly in the token.

3.2. Component Overview

The platform follows a master-worker architecture. The master node handles all management tasks, while worker nodes execute containers and report their status.

In this architecture, there is a clear separation of roles. The master never runs containers itself. It only makes decisions: which node should run a service, when to restart a failed container, whether a user has permission to perform an action. The workers do the actual work: they run Docker containers, monitor their own CPU and RAM usage and report back to the master through heartbeats.

This separation makes the system easier to reason about. If something goes wrong with a container, the problem is on a worker. If something goes wrong with scheduling or permissions, the problem is on the master. Each component has a single job, which makes debugging simpler.

Figure 3.1 shows the complete architecture with all modules and their connections.

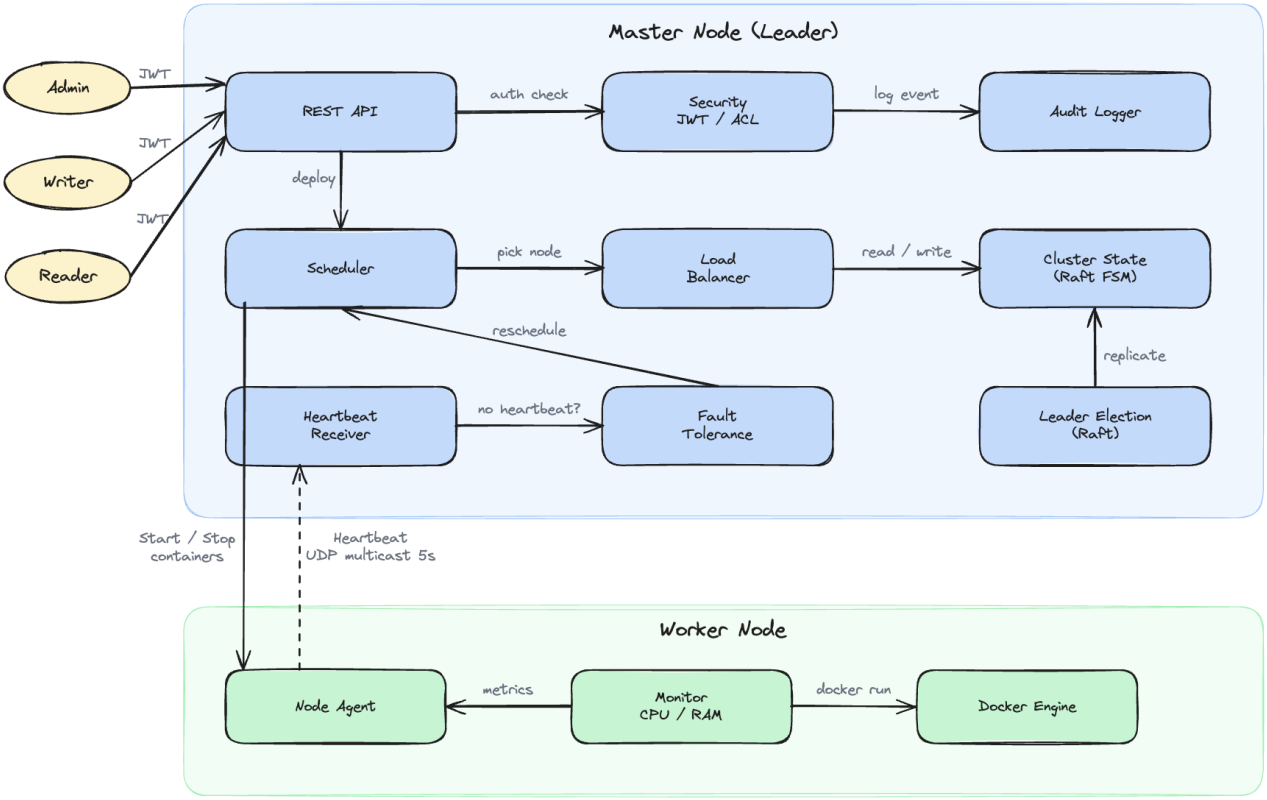


Figura 3.1. Component Overview

3.2.1. Master Node

The master node contains the following modules:
Request pipeline:

- **REST API**. The entry point for users. Exposes endpoints for managing services, nodes and pools.
- **Security**. Handles JWT authentication and ACL authorization. Every request passes through this module before being processed.
- **Audit Logger**. Records all important events in an append-only log.

Scheduling pipeline:

- **Scheduler**. Receives deploy requests and filters nodes in the requested pool that have enough resources. Passes the candidate list to the Load Balancer.
- **Load Balancer**. Selects the best node from the candidate list. Supports two strategies: Least Connections and Weighted.
- **Cluster State (Raft FSM)**. The source of truth for the entire cluster. Stores all data about nodes, services, containers, pools and users. Replicated across 3 master nodes via Raft.

Cluster health:

- **Heartbeat Receiver**. Listens for UDP multicast packets from workers every 5 seconds.
- **Fault Tolerance**. Monitors heartbeats and detects node failures. Marks nodes as SUSPECT after 15 seconds and DEAD after 30 seconds without a heartbeat.
- **Leader Election (Raft)**. Manages leader election across 3 master nodes using the Raft consensus protocol.

3.2.2. Worker Node

Each worker node runs three components:

- **Monitor**. Reads system metrics (CPU usage, RAM usage) from the operating system.
- **Node Agent**. The main process. Sends heartbeats every 5 seconds and listens for commands from the master (StartContainer, KillContainers).
- **Docker Engine**. The agent interacts with Docker to start and stop containers.

The worker is started with the following flags:

- `-id`. Unique node identifier (required)
- `-addr`. TCP address for receiving commands from the master (default: `localhost:7000`)
- `-multicast`. UDP multicast address for heartbeats (default: `239.0.0.1:9090`)
- `-interval`. Heartbeat interval (default: `5s`)

3.2.3. Communication

Master and worker nodes communicate through two channels:

- **Heartbeat**. UDP multicast, every 5 seconds. The worker sends its metrics and the master updates its state. Fire-and-forget, no acknowledgment needed.
- **Commands**. TCP, from master to worker. Used for StartContainer and KillContainers. Only triggered on deploy, delete or fault recovery.

All nodes run within a trusted internal network, so no TLS encryption is used. Both node-to-master traffic (heartbeats and commands) and Raft traffic between master nodes use plain TCP and UDP. Adding TLS would be a natural next step for an untrusted network.

3.3. Domain Model

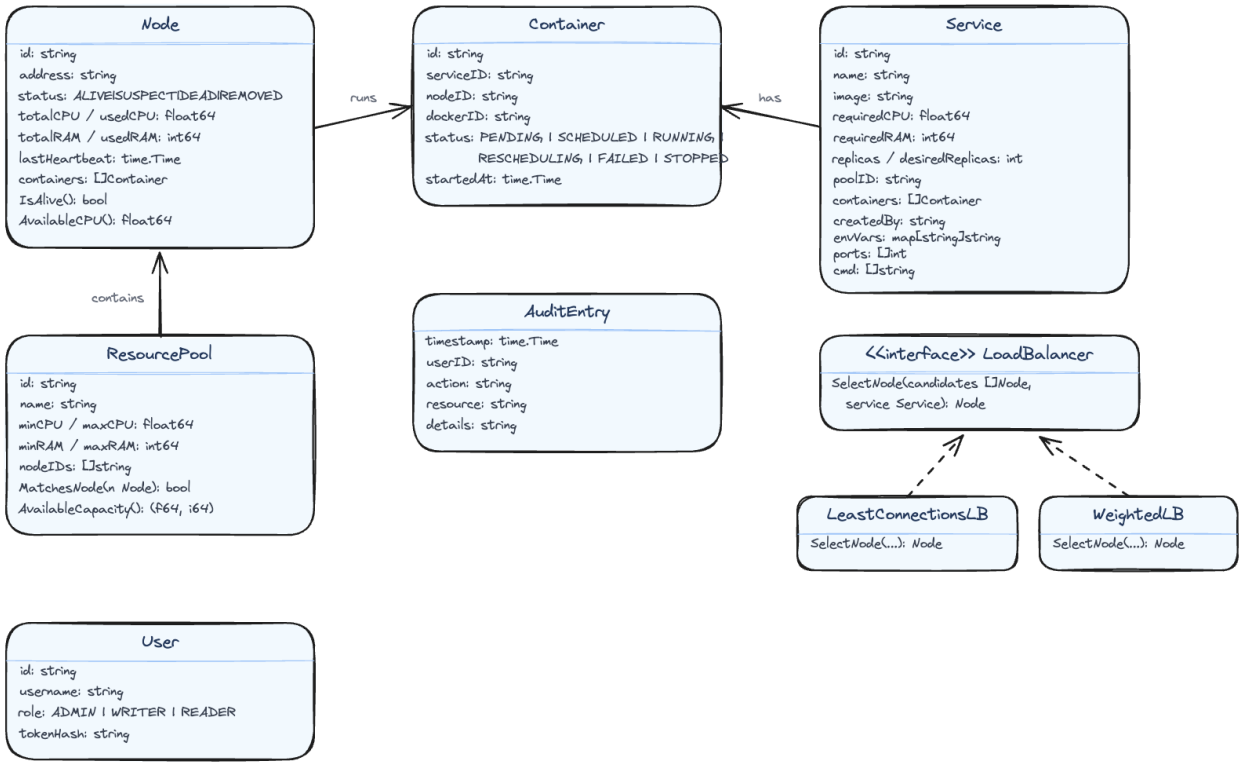


Figura 3.2. Domain Model

The platform uses the following data entities:

Node represents a worker machine in the cluster. It stores the node's address, total and used CPU/RAM, its current status (ALIVE, SUSPECT, DEAD or REMOVED) and the timestamp of the last received heartbeat.

Service represents what the user wants to run. It contains the Docker image name, required CPU and RAM, the number of desired replicas, the target pool and runtime parameters (environment variables, ports, command). These parameters are saved in the cluster state so that containers can be restarted identically during fault recovery.

Container is a running instance of a service on a specific node. It tracks the Docker container ID and its status (PENDING, SCHEDULED, RUNNING, FAILED, RESCHEDULING or STOPPED).

ResourcePool groups nodes with similar hardware. Each pool has CPU and RAM ranges. When a new node is discovered, it is automatically assigned to the matching pool based on its resources. The master creates three default pools at startup: `general` (up to 3 CPU cores), `high-cpu` (4 or more cores) and `high-ram` (more than 32 GB of RAM). The ranges do not overlap, so every node matches exactly one pool. If a node matches no pool, a warning is written in the log and the node cannot receive containers.

User has an ID, username, role (ADMIN, WRITER or READER) and a token hash for authentication.

AuditEntry records a single event with a timestamp, user ID, action, resource and details.

LoadBalancer is an interface with a single method: SelectNode. It has two implementations: LeastConnectionsLB (picks the node with the fewest running containers) and WeightedLB (picks the node with most available resources).

All these entities are stored in an in-memory data structure called `State`, which acts as the single source of truth for the cluster. The State uses Go maps (one per entity type) protected by a read-write mutex (`sync.RWMutex`).

A **goroutine** is a lightweight thread managed by the Go runtime. Unlike regular operating system threads, goroutines are very cheap to create, so a Go program can run thousands of them at the same time. In this platform, the heartbeat receiver, the fault tolerance checker and the API server each run as a separate goroutine. This means they all execute at the same time, in parallel.

The problem with parallel execution is that two goroutines might try to modify the same data at the same time, which can cause data corruption. A **mutex** (mutual exclusion) solves this problem. It works like a lock on a door: only one goroutine can hold the lock at a time. A `RWMutex` is a special type that allows multiple readers to hold the lock at the same time (since reading does not change anything), but only one writer can hold it at a time. This way, reading the cluster state is fast (no waiting), but writing is safe (no corruption).

3.4. Heartbeat and Node Discovery

The heartbeat mechanism is the foundation of the platform. It solves two problems at once: how does the master know which workers exist and how does it know if a worker is still alive. The answer to both questions is the same: the worker sends a small message (heartbeat) every 5 seconds. If the master receives it, the worker is alive. If the master stops receiving it, the worker is probably dead.

This section describes how heartbeats work, how new nodes are discovered and how nodes are removed from the cluster. Figure 3.3 shows the sequence of messages between a worker, the master and the cluster state.

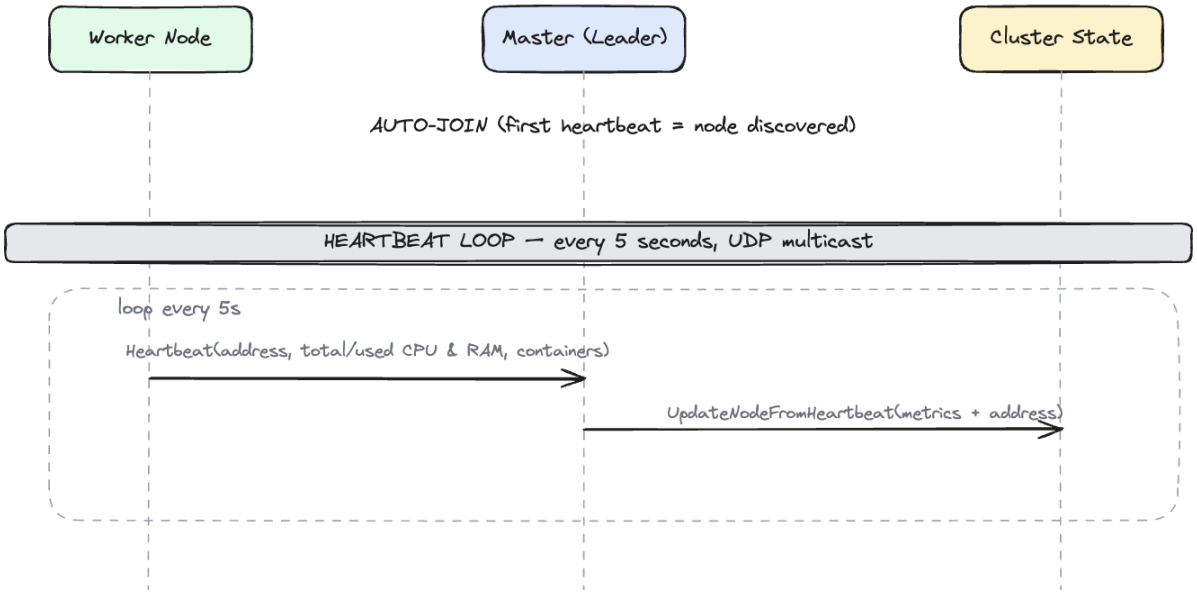


Figura 3.3. Heartbeat and Node Discovery

Node discovery is fully automatic. There is no explicit join or leave request. A worker simply starts sending heartbeats and the master discovers it. When a worker stops, the master notices the missing heartbeats, marks it as dead and reschedules its containers. Explicit join and leave requests would add unnecessary complexity. The heartbeat-based approach is simpler and handles both planned shutdowns and unexpected crashes in the same way.

3.4.1. Auto-Join

When the master receives the first heartbeat from an unknown node, it automatically adds the node to the cluster state (the in-memory data structure described in section 3.3) with status ALIVE and assigns it to the matching resource pool based on the CPU and RAM values reported in the heartbeat. There is no separate database. The live state is kept in memory using Go maps protected by a mutex, while write operations (services and containers) are also stored in the Raft log on disk. When the master restarts without bootstrapping a fresh cluster, it replays the Raft log to rebuild these entries and node entries reappear from the next heartbeats.

3.4.2. Heartbeat Loop

Every 5 seconds, each worker sends a UDP multicast packet containing:

- Total and used CPU (number of cores; usage is approximated from the system load average)
- Total and used RAM (in MB)
- The node's address for receiving TCP commands
- List of running containers

The master receives the packet and updates the node's metrics in the cluster state. No response is sent back because the heartbeat is fire-and-forget. The worker does not wait for an answer and does not care if the master received the packet or not. If a packet is lost, the next one will arrive in 5 seconds.

The metrics sent in the heartbeat are important for two reasons. First, the Scheduler uses them to decide where to place new containers. If a node reports 90% CPU usage, the Scheduler will avoid it and pick a node with more free resources. Second, the Fault Tolerance module uses the timestamp of the last heartbeat to detect dead nodes. If no heartbeat arrives for 30 seconds, the node is considered dead.

3.4.3. Node Removal

There is no explicit leave request. When a worker stops (crash, shutdown or network failure), it simply stops sending heartbeats. The master detects the absence through the fault tolerance mechanism (15 seconds SUSPECT, 30 seconds DEAD), marks the node DEAD and starts fresh containers on other nodes. This means that a planned shutdown and an unexpected crash are handled in exactly the same way. The master does not need to distinguish between the two cases because the result is the same: the node is no longer available and its containers need to be moved elsewhere.

3.5. Fault Tolerance: Detection and Recovery

Fault tolerance is the most complex part of the platform. It handles three scenarios: detecting that a node has failed, restarting the containers that were running on the failed node and preventing duplicate containers when a node that was thought to be dead comes back online. Each scenario requires careful handling to avoid data loss or service interruption.

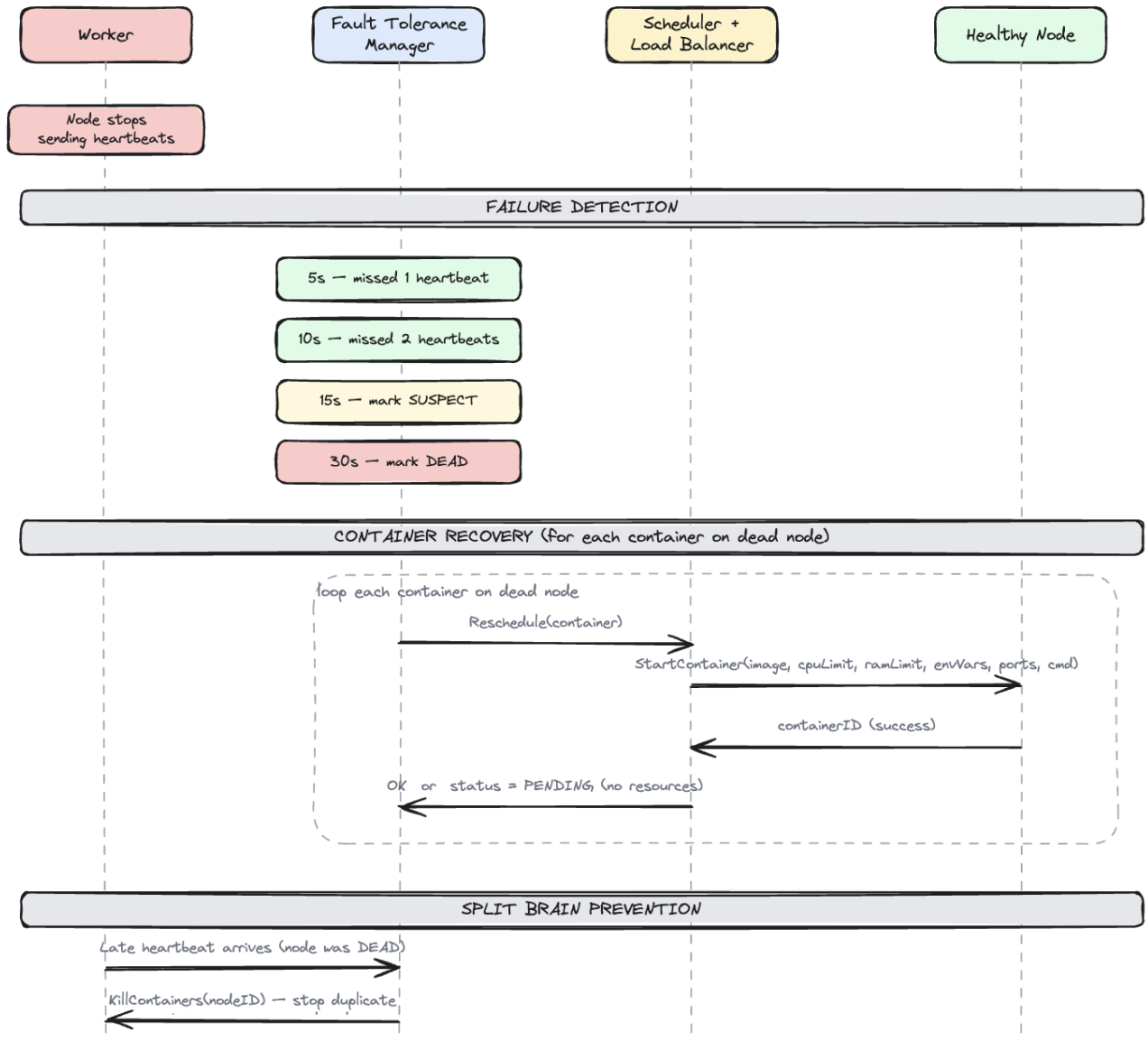


Figura 3.4. Fault Tolerance: Detection and Recovery

3.5.1. Failure Detection

The Fault Tolerance module runs a periodic check (every 5 seconds) on all nodes. For each node, it computes the time since the last heartbeat. After 5 seconds (1 missed heartbeat), no action is taken because it could be network jitter. After **15 seconds** (3 missed heartbeats), the node is marked **SUSPECT**. After **30 seconds** (6 missed heartbeats), the node is marked **DEAD** and recovery begins.

The timing values (15 seconds for SUSPECT and 30 seconds for DEAD) are passed as parameters when creating the Fault Tolerance module. Shorter values make the system react faster to failures but increase the risk of false positives (marking a healthy node as dead because of a temporary network problem). Longer values reduce false positives but mean that containers on a dead node will not be restarted for a longer time.

In the implementation, the check runs as a goroutine with a ticker. Nodes that are already DEAD or REMOVED are skipped.

3.5.2. Container Recovery

When a node is marked DEAD, its services still need to run. The system starts fresh containers on healthy nodes. Recovery only looks at containers that were active on the dead node (RUNNING or SCHEDULED); containers that were already stopped or failed are not restarted. For each active container, the Fault Tolerance module asks the Scheduler to find a new node. The Scheduler filters nodes in the matching pool with enough resources and passes the candidates to the Load Balancer, which picks the best node. A StartContainer command is then sent to the chosen node with the full service parameters (image, CPU limit, RAM limit, environment variables, ports, command). If no node has enough resources, the container stays in PENDING status. A retry loop on the leader checks PENDING containers every 10 seconds. PENDING containers that belong to the same service are started only together (all-or-nothing), matching the deployment semantics; a single container that lost its node during recovery forms a group of one and is retried individually.

The key point is that containers are not moved from the dead node but started fresh from the service parameters saved in the cluster state. This is why the Service entity stores all runtime parameters (image, environment variables, ports, command). The old container on the dead node is marked as STOPPED in the cluster state and a new container entry is created for the replacement.

3.5.3. Split-Brain Prevention

When a node that was marked DEAD comes back online (for example, after a temporary network failure), it sends a heartbeat. The leader detects this as a late heartbeat and sends a `KillContainers` command to the recovered node. This stops all containers on the recovered node, because those containers have already been rescheduled onto other healthy nodes. Without this mechanism, the same containers would run on both the recovered node and the new nodes, creating duplicates.

The order of operations matters here. The node stays in the DEAD state until the worker confirms that the duplicate containers were killed. Only then is the node brought back to ALIVE and made available for new scheduling. If the node were marked ALIVE first, the Scheduler could place a new container on it and the KillContainers command would then destroy that new container by mistake. If the command fails (for example, the worker is not reachable yet), the node stays DEAD and the next heartbeat triggers another attempt. Only the leader sends the command; follower masters simply mark the node ALIVE in their local view.

The worker finds its containers through Docker labels (the node ID and service ID are attached to every container it creates), so it can remove them even if the worker process itself crashed and was restarted in the meantime. After the cleanup, the recovered node is a regular ALIVE node with no containers, ready to accept new work.

The term “split-brain” comes from a scenario where the network is split into two parts and

each part thinks the other is dead. In this platform, split-brain can happen when the master cannot reach a worker (and marks it DEAD) but the worker is still running its containers.

3.6. Worker Node State Machine

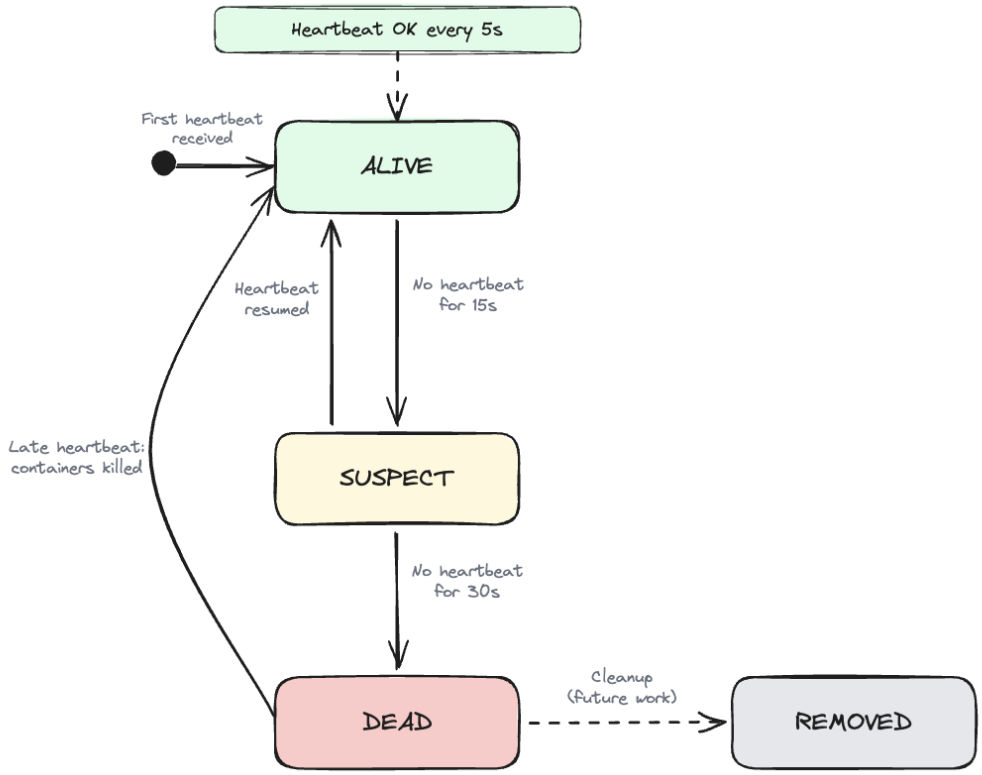


Figura 3.5. Worker Node State Machine

A worker node can be in one of four states:

- **ALIVE**. The initial and normal state. A node enters this state when the master receives its first heartbeat. It sends heartbeats every 5 seconds, runs containers and is available for scheduling.
- **SUSPECT**. The master has not received a heartbeat for 15 seconds. The node might be temporarily unreachable. No action is taken yet.
- **DEAD**. No heartbeat for 30 seconds. The master starts fresh containers on other nodes using the original service parameters.
- **REMOVED**. A reserved final state for a node that has been fully taken out of the cluster. In the current implementation a dead node stays in the DEAD state after its containers are rescheduled; automatic cleanup to REMOVED is left as future work.

The transitions between states are: when the master receives the first heartbeat from a node, it enters the ALIVE state (auto-join). If no heartbeat is received for 15 seconds, the node moves to SUSPECT. If a heartbeat arrives while the node is SUSPECT, it goes back to ALIVE. If no heartbeat is received for 30 seconds, the node moves to DEAD and its containers are rescheduled to other nodes. The dead node stays in the cluster state in the DEAD state; if it later starts sending heartbeats again, it is brought back to ALIVE only after its old containers are confirmed killed (see split-brain prevention).

The SUSPECT state exists to avoid false positives. A single missed heartbeat could be caused by a short network delay, not a real failure.

3.7. Load Balancer

The Load Balancer selects the best node from a list of candidates. It is defined as a Go interface with a single method:

```
type LoadBalancer interface {
    SelectNode(candidates []*Node, service *Service) *Node
}
```

This design allows the load balancing strategy to be swapped without modifying the rest of the code. The platform provides two implementations:

LeastConnectionsLB picks the node with the fewest running containers. The name comes from the classic load balancing strategy; in this platform, the number of running containers is used as the connection count. This strategy works well when all containers have similar resource requirements.

WeightedLB picks the node with the most available resources. The score is computed as a weighted score between available CPU and available RAM. This strategy is better when containers have varying resource needs.

The master can be started with either strategy using the `-lb` flag: `-lb leastconn` (default) or `-lb weighted`.

A custom strategy only needs to satisfy the `SelectNode` method. For example, someone could write a `RandomLB` that picks a random node from the candidate list or a `LocalityLB` that prefers nodes in the same network zone. The Scheduler does not know or care which strategy is used. It just calls `SelectNode` and gets back a node.

The Weighted strategy normalizes both CPU and RAM to a 0 to 1 range (percentage of total resources) before adding them together, so that a node with 8 free CPU cores and 16 GB free RAM is not unfairly favored over a node with 4 free CPU cores and 64 GB free RAM.

3.8. REST API

Every master exposes the REST API (port 8090 by default, configurable via the `-api` flag). The master also accepts `-multicast` to configure the UDP multicast address for heartbeats (default: `239.0.0.1:9090`).

Users do not need to know which master is the current leader. Read requests are answered by any master from its own copy of the state. Write requests (`POST` and `DELETE`) must run on the leader, so a follower that receives one forwards it transparently to the leader's API and returns the leader's response. The forwarding uses the `-peers-api` flag, which gives every master the API address of all masters (`id=host:port,...`); the follower looks up the current leader's ID in this map. A forwarded request is marked with a header so it is never forwarded twice. This way, when the leader dies and a new one is elected, users keep sending requests to the same address as before.

The following endpoints are available:

- `POST /login`. Authenticate with username/password, returns access and refresh tokens
- `POST /refresh`. Exchange a refresh token for a new token pair
- `GET /nodes`. Returns all nodes in the cluster with their status and metrics
- `GET /pools`. Returns all resource pools
- `GET /services`. Returns all deployed services
- `POST /services`. Deploys a new service (triggers scheduling)
- `DELETE /services/{id}`. Deletes a service and stops its containers
- `GET /audit`. Returns audit log entries, with optional filters `?action=...` and `?user=...`

The `POST /services` endpoint accepts a JSON body:

```
{
  "name": "my-app",
  "image": "nginx:latest",
  "cpu": 0.5,
  "ram": 256,
  "replicas": 2,
  "pool": "high-cpu",
  "envVars": {"ENV": "production"},
  "ports": [80],
  "cmd": ["nginx", "-g", "daemon off;"]
}
```

3.9. Service Deployment Flow

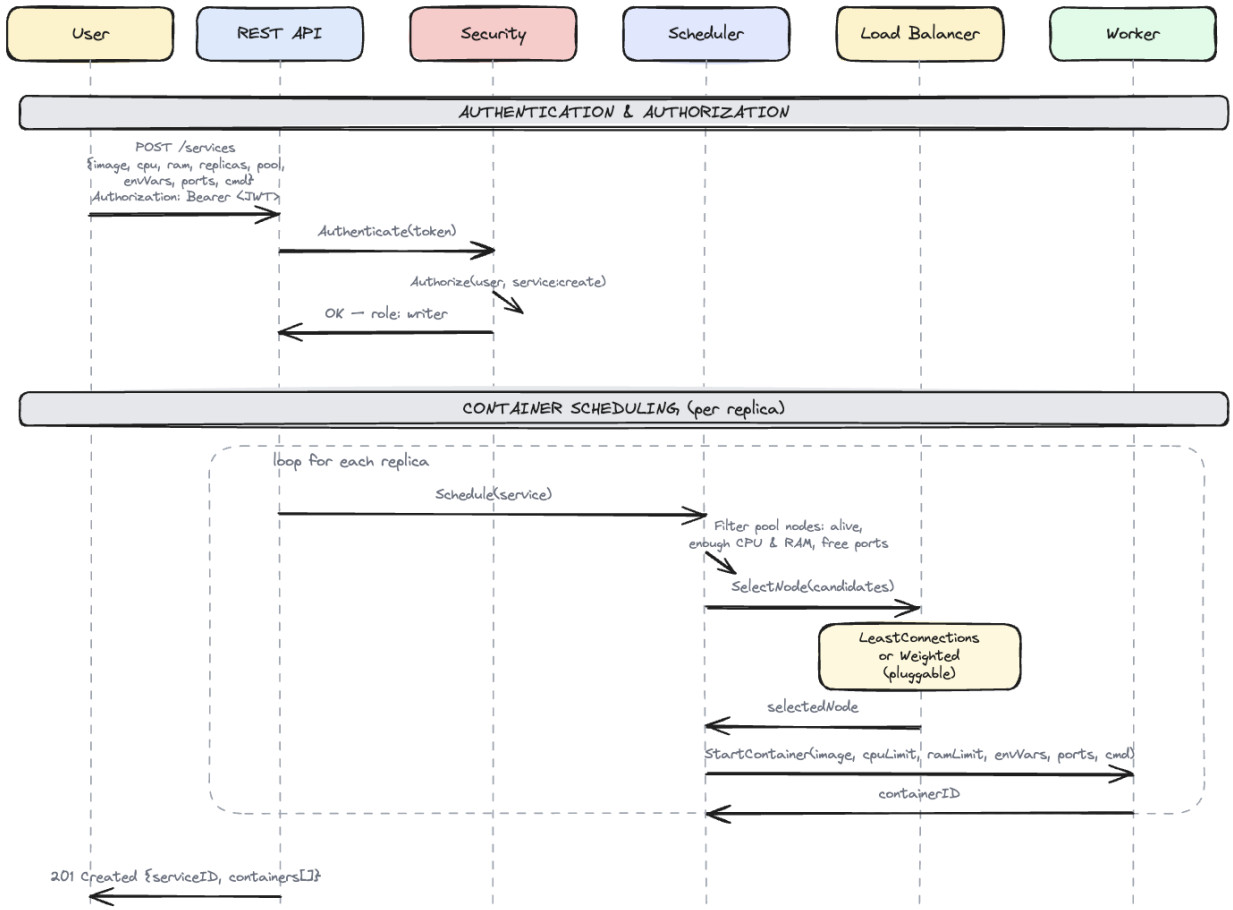


Figura 3.6. Service Deployment Flow

When a user sends a `POST /services` request, the API creates a Service in the cluster state and asks the Scheduler to plan all requested replicas at once. Scheduling is **all-or-nothing**: either every replica gets a node or no container is started.

1. The Scheduler retrieves all nodes in the requested resource pool and builds working copies of them.

2. For each replica, it filters out nodes that are not ALIVE, do not have enough CPU and RAM or already use one of the requested ports (each port can be bound only once per machine) and passes the candidates to the Load Balancer, which picks the best node.
3. The CPU, RAM and ports of the chosen node are updated in the working copy, so the next replica sees the remaining capacity. A single request can therefore never oversubscribe a node.
4. If any replica cannot be placed, nothing is started: all replicas are created with PENDING status and the retry loop starts the whole group later, when the cluster can take all of them.
5. Otherwise, a `StartContainer` command is sent for each replica to its planned node via TCP with the full service parameters.
6. The worker pulls the Docker image, creates and starts the container and returns the container ID.

The API returns a `201 Created` response with the service ID and a list of containers with their statuses and assigned nodes. When the plan fails, the response also contains a message explaining how many replicas could be placed and all containers are reported as PENDING.

This same scheduling flow is used for fault recovery when containers need to be rescheduled after a node dies. The only difference is that during fault recovery, the Fault Tolerance module triggers the scheduling instead of the API. The Scheduler and Load Balancer do not know where the request came from. They just find the best node for the service.

If a service has 3 replicas, the planning loop runs 3 times. Each placement updates the working copy of the chosen node (container count, free CPU and RAM, used ports), so both strategies see the effect of the earlier replicas in the same request. With Least Connections, the replicas spread naturally: the first goes to worker-1 (0 containers), the second to worker-2 (0 containers) and the third back to worker-1. With Weighted, the score of a node drops as replicas are assigned to it, which has the same spreading effect. Between requests, the free resources still come from the last heartbeat, which refreshes every 5 seconds.

When a service is deleted via `DELETE /services/{id}`, the API sends `KillContainers` commands to all workers running that service's containers, marks the containers as STOPPED and removes the service from the cluster state. The KillContainers command includes the service ID so that the worker only stops containers belonging to that service and not all containers on the node.

3.10. Agent: Worker Command Server

Each worker runs a TCP command server that listens for commands from the master. The server accepts two types of commands:

StartContainer. The master sends a JSON command containing all service parameters:

```
{
  "type": "StartContainer",
  "serviceID": "svc-abc123",
  "image": "nginx:latest",
  "cpuLimit": 0.5,
  "ramLimit": 256,
  "envVars": {"ENV": "production"},
  "ports": [80],
  "cmd": ["nginx", "-g", "daemon off;"]
}
```

The worker pulls the Docker image (if not already present), creates a container with the specified resource limits (CPU via `NanoCPUs`, RAM via `Memory`), environment variables, port bindings and command then starts it. The worker responds with:

```
{"success": true, "containerID": "docker-container-id"}
```

KillContainers. The master sends this command during split-brain prevention or when deleting a service. If a `serviceID` is specified, only containers for that service are stopped. If empty, all containers on the worker are stopped.

```
{"type": "KillContainers", "serviceID": "svc-abc123"}
```

Commands are sent as JSON over TCP. The master uses the node's address (reported in heartbeats) to establish a connection, sends the command and waits for a response. This communication is synchronous, meaning the master waits for the worker to confirm before proceeding. If the connection fails (for example because the worker crashed), the master marks the container as PENDING and the retry loop tries again every 10 seconds.

The worker integrates with Docker through the Docker SDK for Go. A `DockerManager` component wraps the Docker client. Every container it creates is tagged with two Docker labels: the worker's node ID and the service ID. The worker finds its own containers by these labels, so the tracking survives a crash and restart of the worker process. The container name has the form `dcp-<nodeID>-<serviceID>-<n>`, which also stays unique when several workers share the same Docker daemon on one machine. If Docker creates a container but fails to start it, the worker removes the created container so it does not leak. When the worker shuts down, it stops and removes all its containers automatically. The list of running container IDs is included in each heartbeat. The master keeps its own record of container placement from the scheduling decisions it makes, so this list is currently informational.

When starting a container, the worker first pulls the Docker image from a registry (like Docker Hub). If the image is already present on the machine, this step is skipped. Then it creates the container with the resource limits (CPU and RAM) that the master specified. The CPU limit is set using Docker's `NanoCPUs` field (0.5 CPU = 500,000,000 nanoseconds of CPU time per second). The RAM limit is set using the `Memory` field in bytes. These limits are enforced by the Linux kernel through a feature called cgroups, which prevents a container from using more resources than it was assigned.

3.11. Container State Machine

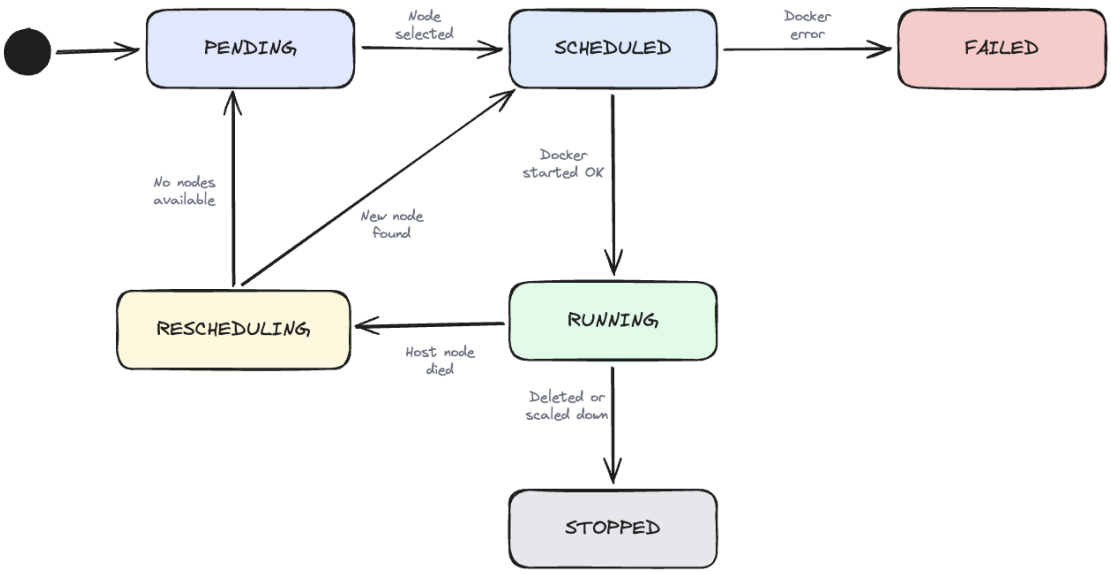


Figura 3.7. Container State Machine

A container goes through the following states:

- **PENDING**. The container needs to run, but the cluster cannot currently take it. The retry loop attempts to schedule it every 10 seconds; PENDING containers of the same service start only together (all-or-nothing).
- **SCHEDULED**. A node was found and the container has been assigned. Docker startup is in progress.
- **RUNNING**. Docker started the container successfully. This is the normal operating state.
- **FAILED**. Docker returned an error (invalid image, port conflict, out of memory). The container is marked FAILED.
- **RESCHEDULING**. The host node died. A fresh container needs to be started on another node.
- **STOPPED**. The container was stopped intentionally when its service was deleted or replaced during fault recovery. This is a final state.

3.12. Security and Audit Architecture

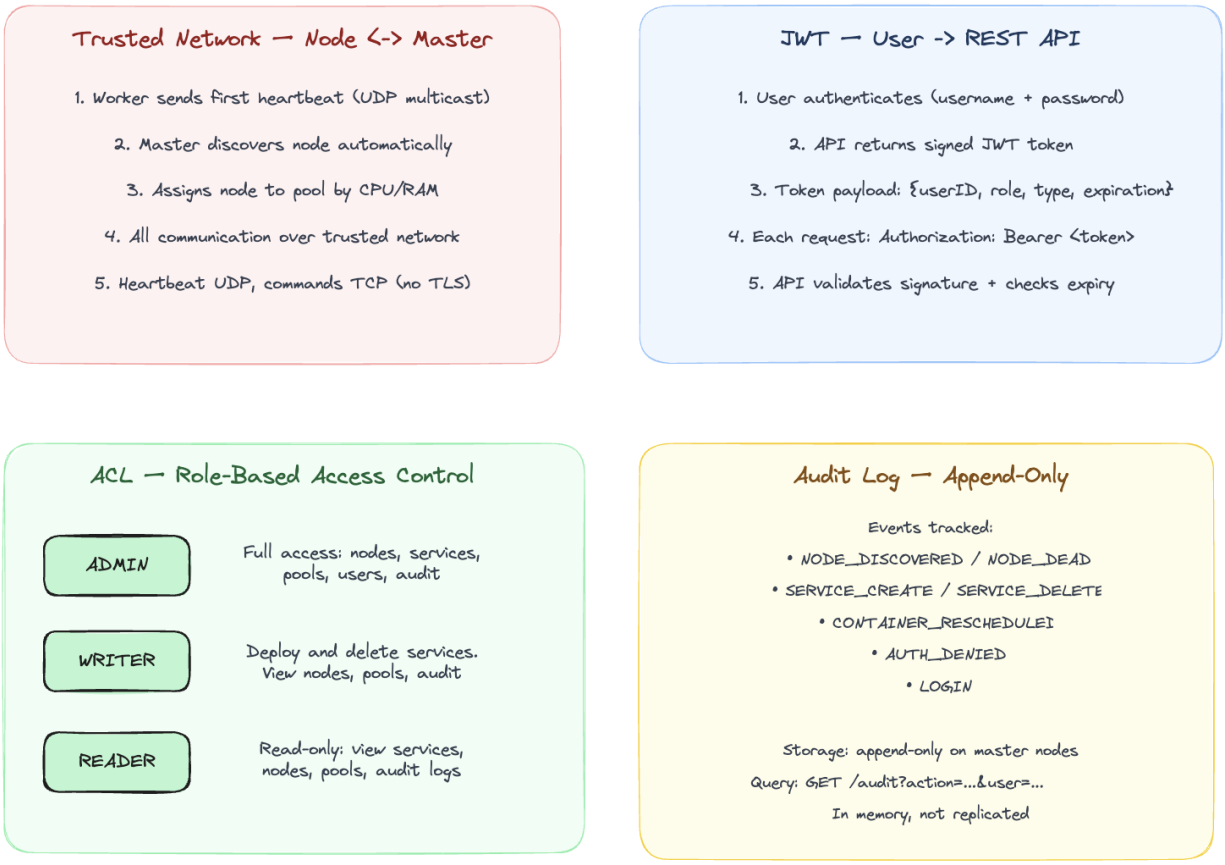


Figura 3.8. Security and Audit Architecture

3.12.1. JWT Authentication with Refresh Tokens

Users authenticate by sending their username and password to the `POST /login` endpoint. The API verifies the credentials against a **SHA-256 hash** stored in the cluster state. SHA-256 is a one-way function: it turns a password like "admin" into a fixed-length string of characters (a hash) like `8c6976e5...`. The important property is that you cannot reverse it, so given the hash you cannot get back the original password. The platform never stores the actual password, only its hash. At login, it hashes the password sent by the user and compares it with the stored hash.

If the credentials are correct, the API returns a token pair:

- **Access token**. A short-lived JWT (15 minutes) containing the user ID, role and expiration. Used in the `Authorization: Bearer <token>` header on every request.
- **Refresh token**. A longer-lived JWT (24 hours) containing only the user ID. Used to obtain a new access token via `POST /refresh` without re-entering credentials.

A **JWT (JSON Web Token)** is a string made of three parts separated by dots: a header, a payload and a signature. The payload contains the user's data (ID, role, expiration time). The signature is created using a secret key known only to the server. When the server receives a JWT, it recalculates the signature and checks if it matches. If someone changed the payload (for example, changed the role from READER to ADMIN), the signature will not match and the token will be rejected.

JWT is **stateless**, which means the server does not need to store any session data. Everything the server needs to know about the user (who they are, what role they have, when the token expires) is inside the token itself. This is different from traditional session-based authentication, where the server stores a session ID in a database and looks it up on every request.

When the access token expires after 15 minutes, the client sends the refresh token to `POST /refresh` and receives a new token pair. This way, the user does not need to enter their password again for 24 hours (the lifetime of the refresh token).

Each token also carries a type field (`access` or `refresh`) that is checked during validation. This prevents one token from being used in place of the other: an access token cannot be sent to `POST /refresh` and a refresh token cannot be used to call the API.

The reason for having two tokens instead of one is security. The access token is sent with every API request, so it is more likely to be intercepted. By keeping it short-lived (15 minutes), the damage from a stolen token is limited. The refresh token is sent only once every 15 minutes to get a new access token, so it is less exposed. If the refresh token is stolen, it can be used for up to 24 hours, but it cannot be used to access the API directly.

In this platform, the secret key used to sign tokens is stored in the master's code. In a production system, it would be stored in a configuration file or a secret manager like HashiCorp Vault.

3.12.2. Role-Based Access Control

The platform has three roles with clearly separated permissions. **ADMIN** has the broadest access: it can create and delete services, manage users and view nodes, pools and audit logs. **WRITER** can deploy and delete services and can view nodes, pools and audit logs but cannot manage users. **READER** has read-only access and can view services, nodes, pools and audit logs but cannot create or delete anything.

Every protected API endpoint passes through an authentication and authorization middleware. First, it extracts the JWT from the `Authorization` header. Then it validates the signature and checks that the token has not expired. Next, it extracts the role from the token claims and checks the ACL table to see if that role has permission for the requested action on the requested resource. If the role does not have permission, the API returns 403 Forbidden and logs an AUTH_DENIED event in the audit log.

The ACL rules are defined in code as a Go map from role name to a list of allowed actions. For example, the ADMIN role has entries like `{Resource: "services", Action: "create"}` and `{Resource: "users", Action: "create"}`, while the READER role only has entries with `Action: "read"`. This makes it easy to see at a glance what each role can and cannot do. Adding a new role or changing permissions requires modifying a single map in the code.

The login and refresh endpoints are public and do not require a token. All other endpoints require a valid access token. If a request is sent without a token or with an expired token, the API returns 401 Unauthorized.

3.13. Leader Election: Raft Consensus

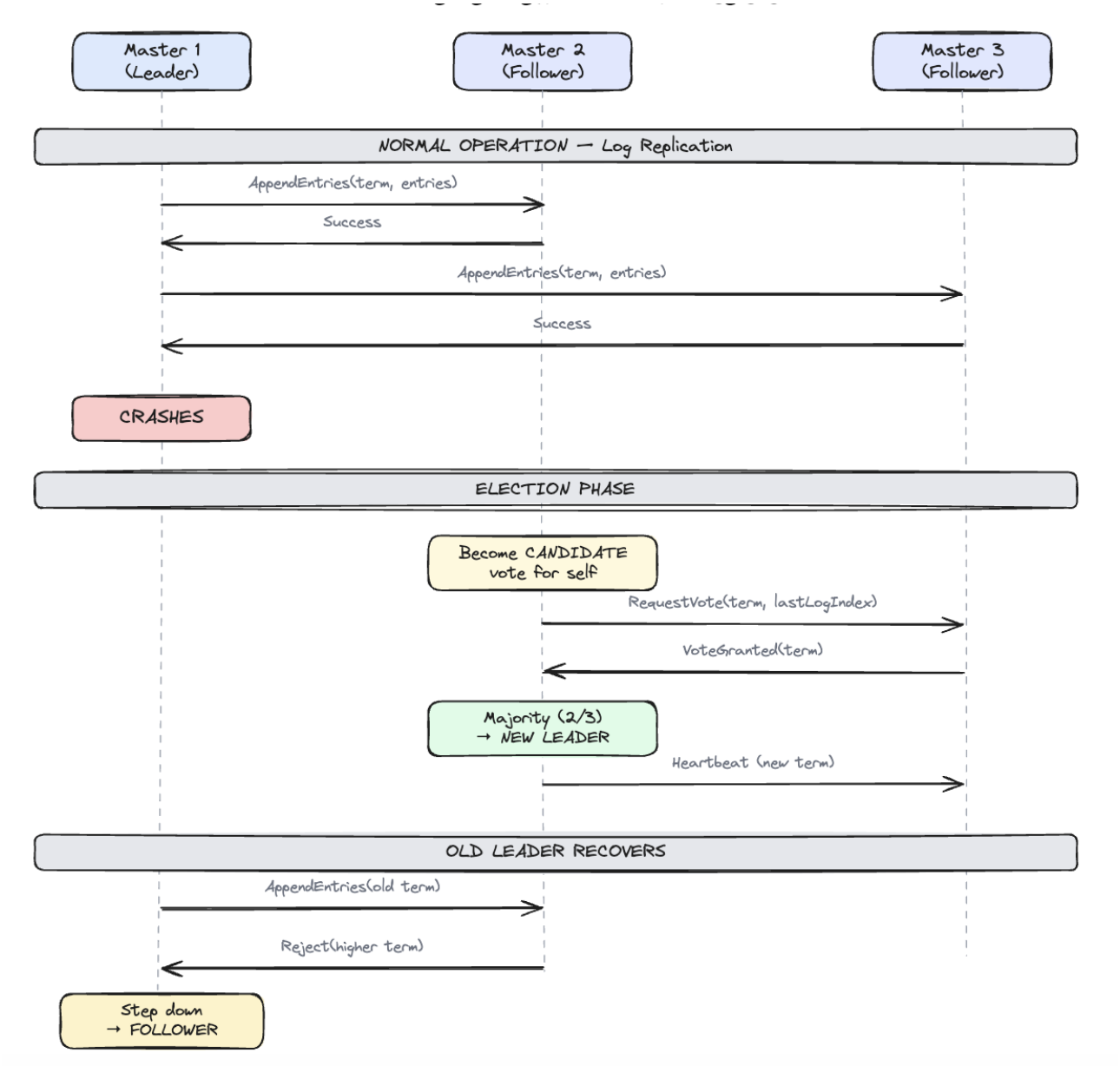


Figura 3.9. Leader Election: Raft Consensus

The platform runs 3 master nodes for high availability. Only one is the leader at any time; the other two are followers that replicate the state.

3.13.1. Normal Operation

The leader sends every state change (node added, service created, container started) to all followers via `AppendEntries` messages. Each follower confirms that it saved the data. This way, every decision is stored on all 3 nodes. The leader waits for a majority of followers to confirm before considering the change committed. With 3 nodes, a majority is 2, so the leader needs only one follower to confirm. This means the cluster can continue working even if one follower is temporarily unreachable.

The followers do not make scheduling or recovery decisions; only the leader does. A write request sent to a follower's API is forwarded transparently to the leader (see the REST API section), so users are not affected by a leader change. Each master, including the followers, still receives

worker heartbeats over UDP multicast and updates its own node list. For replicated data (services and containers), followers only apply what the leader commits through Raft. Their main purpose is to be ready to take over if the leader fails. This keeps all decision-making in one place.

3.13.2. Leader Failure and Election

When the leader crashes, the followers stop receiving Raft heartbeats (these are different from worker heartbeats; Raft heartbeats are sent between master nodes to confirm the leader is alive). Each follower has a random election timeout between 150 and 300 milliseconds. The randomness ensures that not all followers try to become leader at the same time, which would cause a split vote.

When the timeout expires, one follower becomes a candidate. It increments its term number (a counter that goes up with each election), votes for itself and asks the other follower for its vote. The other follower checks if the candidate has up-to-date data (its log is at least as long as the voter's log). If yes, it grants its vote. With 2 votes out of 3 (a majority), the candidate becomes the new leader and starts sending Raft heartbeats to the remaining follower. The entire process takes less than a second.

3.13.3. Old Leader Recovery

When the old leader comes back online, it does not know it has been replaced. It tries to send `AppendEntries` messages with its old term number. The new leader responds with a rejection that includes its higher term number. The old leader sees that its term is outdated, gives up the leader role and becomes a follower. It then syncs its state from the new leader by receiving all the log entries it missed while it was down.

This mechanism guarantees that two leaders can never exist at the same time. The higher term always wins. Even if the old leader comes back before the election is complete, its messages will be rejected because its term is lower.

3.13.4. Implementation

The Raft layer is implemented using the HashiCorp Raft library. Each master node runs a Raft instance with four components. The **Finite State Machine (FSM)** applies committed log entries to the cluster state and supports four operations: AddService, RemoveService, AddContainer and SetContainerStatus. The **BoltDB log store** persists the Raft log to disk, ensuring state survives master restarts. The **TCP transport** handles communication between master nodes. A **snapshot mechanism** periodically serializes the cluster state for faster recovery when a new master joins or an existing one restarts.

The first master node is started with the `-raft-bootstrap` flag to initialize the cluster. It must also know about the other masters through the `-raft-peers` flag, which takes a comma-separated list of peer IDs and addresses in the format `id=host:port`. The other masters are started without `-raft-bootstrap` and will automatically join the cluster when the leader contacts them. The master exposes flags for configuration: `-raft-addr`, `-raft-id`, `-raft-dir`, `-raft-bootstrap`, `-raft-peers` and `-peers-api` (the API addresses of all masters, used to forward write requests to the leader).

Each master stores its Raft data in a separate directory (`-raft-dir`), which contains the BoltDB log file (`raft.db`) and periodic snapshots. When a master restarts, Raft reads the log from disk and replays it through the FSM to reconstruct the cluster state.

State changes that go through Raft (AddService, RemoveService, AddContainer, SetContainerStatus) are written to the cluster state only by the FSM. Neither the API nor the fault recovery modifies replicated data directly; both submit log entries through the leader. This ensures that all masters apply the same operations in the same order. Heartbeats and node metrics do not go through Raft because they are ephemeral and each master receives them independently via UDP multicast and updates its local state every 5 seconds.

3.13.5. Cluster Deployment Topology

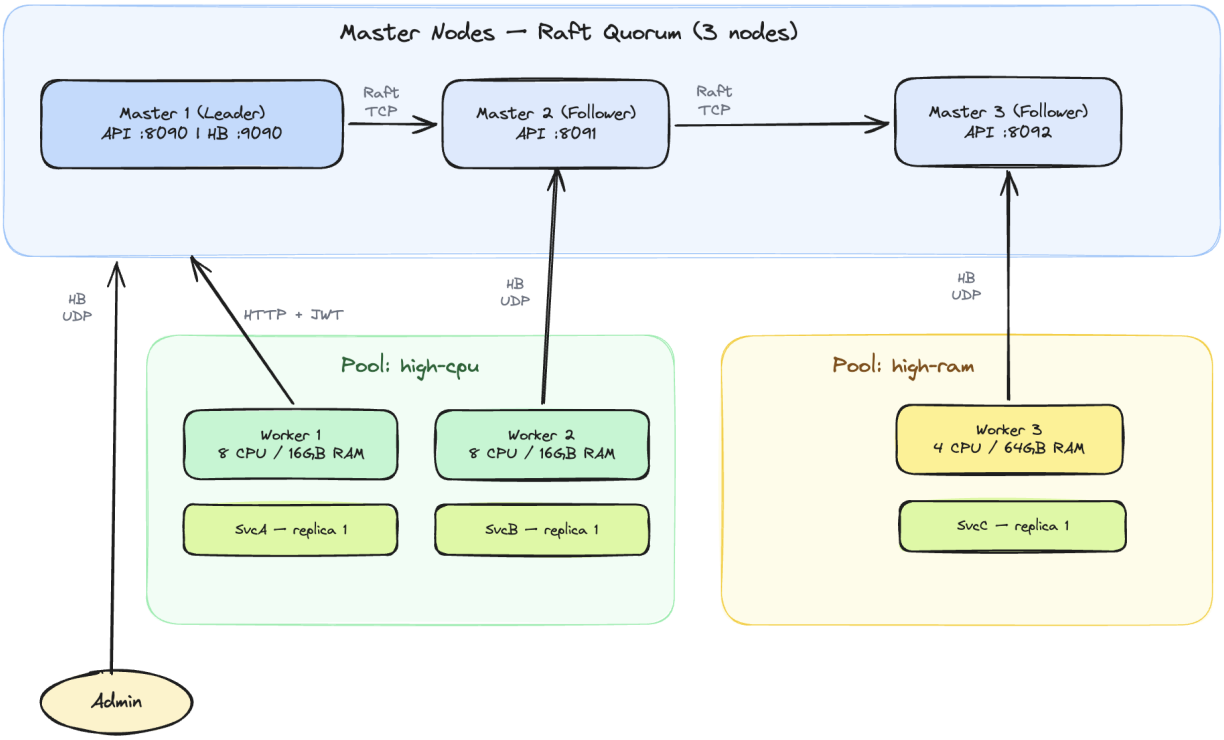


Figura 3.10. Cluster Deployment Topology

The cluster consists of 3 master nodes forming a Raft quorum. Master 1 is the leader. Every master exposes the REST API (ports 8090–8092) and listens for heartbeats on UDP port 9090; write requests sent to a follower are forwarded to the leader. Masters 2 and 3 replicate state via Raft. Worker nodes are organized in resource pools and send heartbeats via UDP multicast to the master. Users connect to the master's REST API using JWT authentication.

3.13.6. Audit Log

All important actions are recorded in an append-only audit log stored in the in-memory cluster state on each master. The log is not replicated through Raft, so each master keeps its own record of the events it observes. The following events are tracked:

- `NODE_DISCOVERED`. When a new node is detected via its first heartbeat
- `NODE_DEAD`. When a node is marked DEAD after 30 seconds without heartbeat
- `SERVICE_CREATE`. When a new service is deployed
- `SERVICE_DELETE`. When a service is deleted
- `CONTAINER_RESCHEDULED`. When a container is moved to another node during fault recovery
- `AUTH_DENIED`. When a user attempts an action they do not have permission for
- `LOGIN`. When a user authenticates successfully

The audit log can be queried through the `GET /audit` endpoint with optional filters for event type (`?action=...`) and user (`?user=...`).

3.14. Implementation

The platform is structured as a single Go module with two binaries: `master` and `worker`. The following packages have been implemented:

- `internal/domain`. Data models (Node, Service, Container, ResourcePool, User, AuditEntry, Heartbeat)
- `internal/cluster`. In-memory cluster state with thread-safe CRUD operations, Raft FSM for state replication, Raft node setup with BoltDB log store and TCP transport
- `internal/heartbeat`. UDP multicast sender (worker) and receiver (master) with auto-join on first heartbeat
- `internal/fault`. Fault tolerance: periodic health checks, SUSPECT/DEAD detection, KillContainers for split-brain prevention
- `internal/loadbalancer`. LoadBalancer interface with LeastConnections and Weighted implementations
- `internal/scheduler`. Filters nodes by pool and resources, delegates to Load Balancer for node selection
- `internal/api`. REST API server with JWT authentication middleware, endpoints for login, refresh, nodes, pools, services
- `internal/security`. JWT token generation/validation (access + refresh tokens), ACL permission checks
- `internal/audit`. Append-only audit logger, integrated with API, heartbeat receiver and fault tolerance
- `internal/agent`. TCP command server (StartContainer, KillContainers), Docker integration via Docker SDK (pull, create, start, stop, remove containers), system metrics monitor (CPU, RAM) and command client for master-to-worker communication

3.15. How to Run

The platform requires Go 1.21+ and Docker to be installed. To build the binaries:

```
go build -o bin/master ./cmd/master
go build -o bin/worker ./cmd/worker
```

To start a cluster with a single master, open three separate terminals:

```
# Terminal 1: start the master
./bin/master -raft-bootstrap

# Terminal 2: start a worker
./bin/worker -id worker-1 -addr localhost:7001

# Terminal 3: start another worker
./bin/worker -id worker-2 -addr localhost:7002
```

To start a cluster with 3 masters for high availability, open five terminals. The first master must know about the other two through the `-raft-peers` flag:

```
# API address map, shared by all masters
PEERS_API="master-1=localhost:8090,\
master-2=localhost:8091,master-3=localhost:8092"

# Terminal 1: master-1 (leader, knows about peers)
./bin/master -raft-bootstrap \
  -raft-id master-1 -raft-addr localhost:9001 \
  -raft-dir /tmp/dcp-raft/m1 -api :8090 \
```

```
  -raft-peers "master-2=localhost:9002,master-3=localhost:9003" \
  -peers-api "$PEERS_API"

# Terminal 2: master-2 (follower)
./bin/master -raft-id master-2 \
  -raft-addr localhost:9002 \
  -raft-dir /tmp/dcp-raft/m2 -api :8091 \
  -peers-api "$PEERS_API"

# Terminal 3: master-3 (follower)
./bin/master -raft-id master-3 \
  -raft-addr localhost:9003 \
  -raft-dir /tmp/dcp-raft/m3 -api :8092 \
  -peers-api "$PEERS_API"

# Terminal 4: worker
./bin/worker -id worker-1 -addr localhost:7001
```

Once all masters are running, master-1 wins the election and becomes leader. If master-1 is stopped, one of the remaining masters will win a new election and become leader within 1 second. The worker continues to send heartbeats to all masters via UDP multicast, so all masters see the same nodes. Requests can be sent to any master's API (8090, 8091 or 8092): reads are answered locally and writes are forwarded to the current leader, so the same address keeps working after a leader change.

The master starts listening for heartbeats on UDP port 9090 and serves the REST API on TCP port 8090. Workers are discovered automatically within 5 seconds. To deploy a service:

```
# Login
curl -X POST http://localhost:8090/login \
  -d '{"username":"admin","password":"admin"}'

# Deploy (use the accessToken from the login response)
curl -X POST http://localhost:8090/services \
  -H "Authorization: Bearer <TOKEN>" \
  -d '{"name":"nginx","image":"nginx:latest",
       "cpu":0.5,"ram":256,"replicas":1,
       "pool":"high-cpu"}'
```

The login response contains an `accessToken` that must be placed in the `Authorization` header of every following request. The deploy response returns the service ID and, for each replica, the container ID, the node it was placed on and its status.

Once one or more services are running, the read endpoints show the state of the cluster. They require only a valid token (any role, including READER) and can be sent to any master:

```
# List all worker nodes with their status and live metrics
curl -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8090/nodes

# List the resource pools and the nodes assigned to each
curl -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8090/pools
```

```
# List the deployed services and their running replica count
curl -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8090/services

# Read the audit log, optionally filtered by action or user
curl -H "Authorization: Bearer <TOKEN>" \
  "http://localhost:8090/audit?action=SERVICE_CREATE"
```

The `/nodes` endpoint is the quickest way to confirm that the cluster works: each worker appears with status ALIVE, its total and used CPU and RAM and the containers currently running on it. After deploying a service, the same call shows the new container in the RUNNING state on the chosen node. A service is removed, together with its containers, with a single request:

```
# Delete a service and stop all its containers
curl -X DELETE \
  -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8090/services/<serviceID>
```

To see fault tolerance in action, stop a worker that runs containers (for example with Ctrl+C). After about 30 seconds the master marks it DEAD and a call to `/nodes` shows its containers restarted on another worker. When the stopped worker is started again, the master sends it a KillContainers command so that no duplicate containers remain.

Capitolul 4. Testing and Experimental Results

This chapter presents the testing methodology and results. The platform was tested at two levels: automated unit tests for individual components and manual end-to-end tests to verify the complete workflow.

The purpose of testing is to verify that each component works correctly on its own (unit tests) and that all components work together as a complete system (manual tests). Unit tests are fast, repeatable and can be run automatically. Manual tests are slower but they test real scenarios with actual Docker containers, network communication and user interaction.

The test environment consists of a single macOS machine (Apple Silicon, 11 CPU cores, 18 GB RAM) running Docker Desktop. The master and workers run as separate processes on the same machine, communicating through localhost. One limitation of this setup is that all workers share the same Docker daemon and the same host ports. The platform handles this by including the worker ID in container names and labels, but two containers that bind the same port can still collide at the Docker level when their workers run on one machine, because the scheduler sees the workers as separate nodes. On separate physical machines this situation cannot occur.

4.1. Unit Tests

Unit tests verify that individual components work correctly in isolation. Each test creates the needed objects in memory, calls a function and checks that the result is correct. The tests are written in Go using the standard `testing` package and can be run with `go test ./...`. No external services (Docker, network) are needed for unit tests.

4.1.1. Load Balancer Tests

Four tests verify the LeastConnections and Weighted strategies:

- **TestLeastConnections_SelectsNodeWithFewestContainers**. Given 3 nodes with 5, 2 and 8 containers respectively, the load balancer selects the node with 2 containers.
- **TestLeastConnections_EmptyCandidates**. When no candidates are provided, returns nil.
- **TestWeighted_SelectsNodeWithMostResources**. Given 3 nodes with different CPU/RAM usage, the load balancer selects the node with the highest available resources (87% CPU free, 87% RAM free).
- **TestWeighted_EmptyCandidates**. When no candidates are provided, returns nil.

4.1.2. Scheduler Tests

Four tests verify the scheduling logic:

- **TestSchedule_FindsNode**. With one alive node that has enough resources, the scheduler returns it.
- **TestSchedule_NotEnoughResources**. When the service requires 100 CPU cores but the node only has 6 available, the scheduler returns an error.
- **TestPlan_AllOrNothing**. The node has capacity for one replica of 4 CPU but not for two: planning one replica succeeds, planning two replicas fails and no node is assigned.
- **TestSchedule_EmptyPool**. When the pool has no nodes, the scheduler returns an error.

4.1.3. ACL Tests

Four tests verify the role-based access control:

- **TestAdminCanDoEverything**. Admin can create, delete and read services, nodes, users and audit.
- **TestReaderCannotCreate**. Reader cannot create or delete services.
- **TestWriterCannotManageUsers**. Writer cannot create users.
- **TestInvalidRole**. An unknown role has no permissions.

4.1.4. Results

All unit tests pass:

```
$ go test ./internal/loadbalancer/ ./internal/scheduler/ \
          ./internal/security/ -v

--- PASS: TestLeastConnections_SelectsNodeWithFewestContainers
--- PASS: TestLeastConnections_EmptyCandidates
--- PASS: TestWeighted_SelectsNodeWithMostResources
--- PASS: TestWeighted_EmptyCandidates
ok   internal/loadbalancer

--- PASS: TestSchedule_FindsNode
--- PASS: TestSchedule_NotEnoughResources
--- PASS: TestPlan_AllOrNothing
--- PASS: TestSchedule_EmptyPool
ok   internal/scheduler

--- PASS: TestAdminCanDoEverything
--- PASS: TestReaderCannotCreate
--- PASS: TestWriterCannotManageUsers
--- PASS: TestInvalidRole
ok   internal/security
```

4.2. Manual Testing

The following scenarios were tested manually to verify the end-to-end behavior of the platform. Each test follows the same structure: an objective describing what is being verified, the steps to reproduce the test, the expected result with specific details about what the output should look like and a screenshot showing the actual result.

4.2.1. Node Discovery and Heartbeats

Objective: Verify that worker nodes are discovered automatically when they start sending heartbeats and that the master correctly displays their status and metrics.

Steps:

1. Start the master: `./bin/master -raft-bootstrap`
2. Start worker-1: `./bin/worker -id worker-1 -addr localhost:7001`
3. Start worker-2: `./bin/worker -id worker-2 -addr localhost:7002`
4. Wait 10 seconds and observe the master log.

Expected result: The master log shows “new node discovered” for each worker within 5 seconds of starting. The cluster status display (every 10 seconds) lists both workers with status ALIVE,

their CPU and RAM usage and the time since their last heartbeat. This confirms that auto-join works and that heartbeats are being received correctly.

```
2026/05/26 10:35:21 api: listening on :8090
2026-05-26T10:35:23.279+0300 [WARN]  raft: heartbeat timeout reached, starting election: last-leader-addr= last-leader-id=
2026-05-26T10:35:23.279+0300 [INFO]  raft: entering candidate state: node="Node at 127.0.0.1:9000 [Candidate]" term=2
2026-05-26T10:35:23.279+0300 [DEBUG] raft: pre-voting for self: term=2 id=master-1
2026-05-26T10:35:23.279+0300 [DEBUG] raft: calculated votes needed: needed=1 term=2
2026-05-26T10:35:23.279+0300 [DEBUG] raft: pre-vote received: from=master-1 term=2 tally=0
2026-05-26T10:35:23.279+0300 [DEBUG] raft: pre-vote granted: from=master-1 term=2 tally=1
2026-05-26T10:35:23.279+0300 [INFO]  raft: pre-vote successful, starting election: term=2 tally=1 refused=0 votesNeeded=1
2026-05-26T10:35:23.286+0300 [DEBUG] raft: voting for self: term=2 id=master-1
2026-05-26T10:35:23.298+0300 [DEBUG] raft: vote granted: from=master-1 term=2 tally=1
2026-05-26T10:35:23.298+0300 [INFO]  raft: election won: term=2 tally=1
2026-05-26T10:35:23.298+0300 [INFO]  raft: entering leader state: leader="Node at 127.0.0.1:9000 [Leader]"
2026/05/26 10:35:31 cluster status: no nodes
2026/05/26 10:35:36 heartbeat receiver: new node discovered: worker-1 (localhost:7001) CPU=11.0 RAM=8MB
2026/05/26 10:35:36 audit: [NODE_DISCOVERED] user=system resource=worker-1 details=node discovered via first heartbeat
2026/05/26 10:35:36 fsm: apply AddNode worker-1
2026/05/26 10:35:40 heartbeat receiver: new node discovered: worker-2 (localhost:7002) CPU=11.0 RAM=8MB
2026/05/26 10:35:40 audit: [NODE_DISCOVERED] user=system resource=worker-2 details=node discovered via first heartbeat
2026/05/26 10:35:40 fsm: apply AddNode worker-2
2026/05/26 10:35:41 cluster status: 2 node(s)
2026/05/26 10:35:41   worker-2 [ALIVE] CPU=1.1/11.0 RAM=0/8MB last_hb=1s ago
2026/05/26 10:35:41   worker-1 [ALIVE] CPU=1.1/11.0 RAM=0/8MB last_hb=0s ago
```

Figura 4.1. Node discovery: two workers auto-joined the cluster

4.2.2. Service Deployment

Objective: Verify that a user can authenticate, deploy a service and that the container is actually running on Docker.

Steps:

1. Login as admin:

```
curl -X POST http://localhost:8090/login \
  -d '{"username":"admin","password":"admin"}'
```

2. Deploy nginx using the access token from step 1:

```
curl -X POST http://localhost:8090/services \
  -H "Authorization: Bearer <TOKEN>" \
  -d '{"name":"nginx","image":"nginx:latest","cpu":0.5,
       "ram":256,"replicas":1,"pool":"high-cpu"}'
```

3. Verify the container is running: `docker ps`

Expected result: The login returns a JSON with `accessToken`, `refreshToken` and `expiresIn` (900 seconds). The deploy returns 201 Created with a `serviceID` and a `containers` array where each container has status RUNNING and a `nodeID`. The `docker ps` command shows a container named `dcp-<workerID>-<serviceID>-<n>` running on the machine.

```
george@MacBook distributed_cluster_platform %  curl -s -X POST http://localhost:8090/login -d '{"username":"admin","password":"admin"}'
{"accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MSIsInJvbGUiOiJBRE1JTiIsImV4cCI6MTc3OTc4NTQ3OSwiaWF0IjoxNzc5Nzg0NTc5fQ.jn-FoiMsuEhkqpoAlqce3j8_ODo-qp2ySAg5LnkhqyU","refreshToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MSIsImV4cCI6MTc3OTg3MDk3OSwiaWF0IjoxNzc5Nzg0NTc5fQ.5iCbg5_5BRR1ZvBrMIxTiUxsoourryr1RU99loz21E0
","expiresIn":900}

george@MacBook distributed_cluster_platform % curl -s -X POST http://localhost:8090/services -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MS
IsInJvbGUiOiJBRE1JTiIsImV4cCI6MTc3OTc4NTQ3OSwiaWF0IjoxNzc5Nzg0NTc5fQ.jn-FoiMsuEhkqpoAlqce3j8_ODo-qp2ySAg5LnkhqyU" -d '{"name":"nginx","image":"nginx:latest","cpu":0.5,"ram":2
56,"replicas":1,"pool":"high-cpu"}'
{"containers":[{"id":"ctr-dmgavjqt","nodeID":"worker-1","status":"RUNNING"}],"name":"nginx","serviceID":"svc-fquubrsk"}

george@MacBook distributed_cluster_platform % docker ps --filter "name=dcp-"
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS     NAMES
75e752df3322   nginx:latest   "/docker-entrypoint.…"   About a minute ago   Up About a minute   80/tcp    dcp-svc-fquubrsk
```

Figura 4.2. Service deployment: nginx container running on worker-1

4.2.3. Fault Tolerance

Objective: Verify that the master detects a failed node, transitions it through SUSPECT and DEAD states and handles split-brain when the node recovers.

Steps:

1. With both workers running, stop worker-1 with Ctrl+C.
2. Wait 15 seconds and observe SUSPECT in the master log.
3. Wait 30 seconds and observe DEAD in the master log.
4. Restart worker-1: `./bin/worker -id worker-1 -addr localhost:7001`
5. Observe KillContainers in the master and worker logs.

Expected result: After 15 seconds without heartbeat, the master log shows "node worker-1 marked SUSPECT". After 30 seconds, it shows "node worker-1 marked DEAD". When worker-1 is restarted, the master log shows "late heartbeat from DEAD node worker-1, sending KillContainers" followed by "node worker-1 back to ALIVE". The worker log confirms it received the KillContainers command. This verifies the complete fault tolerance cycle.

```
2026/05/26 11:39:18   worker-1 [ALIVE] CPU=7.3/11.0 RAM=7244/18432MB last_hb=12s ago
2026/05/26 11:39:18   worker-2 [ALIVE] CPU=7.1/11.0 RAM=6989/18432MB last_hb=5s ago
2026/05/26 11:39:23 fault tolerance: node worker-1 marked SUSPECT (no heartbeat for 17s)
2026/05/26 11:39:28 cluster status: 2 node(s)
2026/05/26 11:39:28   worker-2 [ALIVE] CPU=6.9/11.0 RAM=6961/18432MB last_hb=5s ago
2026/05/26 11:39:28   worker-1 [SUSPECT] CPU=7.3/11.0 RAM=7244/18432MB last_hb=22s ago
2026/05/26 11:39:38 fault tolerance: node worker-1 marked DEAD (no heartbeat for 32s)
2026/05/26 11:39:38 recovery: node worker-1 died with 1 container(s)
2026/05/26 11:39:38 audit: [NODE_DEAD] user=system resource=worker-1 details=node died with 1 containers
2026/05/26 11:39:38 scheduler: selected node worker-2 for service nginx (4.0 CPU, 11619 MB RAM available)
2026/05/26 11:39:38 cluster status: 2 node(s)
2026/05/26 11:39:38   worker-1 [DEAD] CPU=7.3/11.0 RAM=7244/18432MB last_hb=32s ago
2026/05/26 11:39:38   worker-2 [ALIVE] CPU=7.0/11.0 RAM=6813/18432MB last_hb=5s ago
2026/05/26 11:39:39 audit: [CONTAINER_RESCHEDULED] user=system resource=ctr-dmgavjqt details=rescheduled to node worker-2 as ctr-1779784779840751000
2026/05/26 11:39:39 recovery: container ctr-dmgavjqt rescheduled to node worker-2 as ctr-1779784779840751000
2026/05/26 11:39:48 cluster status: 2 node(s)
2026/05/26 11:39:48   worker-1 [DEAD] CPU=7.3/11.0 RAM=7244/18432MB last_hb=42s ago
2026/05/26 11:39:48   worker-2 [ALIVE] CPU=7.9/11.0 RAM=7454/18432MB last_hb=5s ago
2026/05/26 11:39:51 fault tolerance: late heartbeat from DEAD node worker-1, sending KillContainers
2026/05/26 11:39:51 fault tolerance: node worker-1 back to ALIVE (containers killed, available for scheduling)
2026/05/26 11:39:58 cluster status: 2 node(s)
2026/05/26 11:39:58   worker-1 [ALIVE] CPU=6.7/11.0 RAM=7093/18432MB last_hb=1s ago
2026/05/26 11:39:58   worker-2 [ALIVE] CPU=7.2/11.0 RAM=7081/18432MB last_hb=5s ago
```

Figura 4.3. Fault tolerance: node marked SUSPECT, then DEAD, then recovered

4.2.4. Security: Role-Based Access Control

Objective: Verify that the READER role cannot create services while the ADMIN role can.
Steps:

1. Login as reader:

```
curl -X POST http://localhost:8090/login \
  -d '{"username":"reader","password":"reader"}'
```

2. Try to deploy a service using the reader token.
3. Login as admin and deploy the same service.

Expected result: The reader login succeeds but the deploy request returns 403 Forbidden with the error "role READER cannot create services". The master log shows an AUTH_DENIED audit event. The admin login and deploy succeed with 201 Created. This confirms that the ACL middleware correctly enforces role-based permissions.

```
george@MacBook distributed_cluster_platform %  curl -s -X POST http://localhost:8090/login -d '{"username":"reader","password":"reader"}'
{"accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MyIsInJvbGUiOiJSRUFERVIiLCJleHAiOjE3Nzk3ODU3NTksImlhdCI6MTc3OTc4NDg1OX0.
p04Vy4_wi8","refreshToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MyIsImV4cCI6MTc3OTg3MTI1OSwiaWF0IjoxNzc5Nzg0ODU5fQ.NiLAgrPs1
8","expiresIn":900}

george@MacBook distributed_cluster_platform % curl -s -X POST http://localhost:8090/services -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsIn
IsInJvbGUiOiJSRUFERVIiLCJleHAiOjE3Nzk3ODU3NTksImlhdCI6MTc3OTc4NDg1OX0.Kiiuhv-Tjm7DT7vcRgxJqIFtUSNsVI5qep04Vy4_wi8" -d '{"name":"test","image"
licas":1,"pool":"high-cpu"}'

{"error":"role READER cannot create services"}
```

Figura 4.4. Security: reader denied, admin allowed

4.2.5. Audit Log

Objective: Verify that all important actions are recorded in the audit log and that the log can be queried with filters.

Steps:

1. Perform several actions: login, deploy a service, delete a service.
2. Query the full audit log:

```
curl -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8090/audit
```

3. Query with a filter:

```
curl -H "Authorization: Bearer <TOKEN>" \
  "http://localhost:8090/audit?action=LOGIN"
```

Expected result: The audit log returns a JSON array of events. Each event has a timestamp, user ID, action type (LOGIN, SERVICE_CREATE, NODE_DISCOVERED, AUTH_DENIED, etc.) and the affected resource. The filtered query returns only events matching the specified action. The log is append-only, so all events from the current session are present in chronological order.

```
george@MacBook distributed_cluster_platform %  curl -s -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MSIsInJvbGUiOiJBRE1JT
NjE5MywiaWF0IjoxNzc5Nzg1MjkzfQ.06Ifkw58VPzrrpqWr3O-zrj8bmOA_QJmHMd-SKIhLpA" http://localhost:8090/audit | python3 -m json.tool
[
    {
        "Timestamp": "2026-05-26T11:36:15.666529+03:00",
        "UserID": "system",
        "Action": "NODE_DISCOVERED",
        "Resource": "worker-1",
        "Details": "node discovered via first heartbeat"
    },
    {
        "Timestamp": "2026-05-26T11:36:18.304646+03:00",
        "UserID": "system",
        "Action": "NODE_DISCOVERED",
        "Resource": "worker-2",
        "Details": "node discovered via first heartbeat"
    },
    {
        "Timestamp": "2026-05-26T11:36:19.930504+03:00",
        "UserID": "u1",
        "Action": "LOGIN",
        "Resource": "auth",
        "Details": "user admin logged in"
    },
    {
        "Timestamp": "2026-05-26T11:37:00.772522+03:00",
        "UserID": "",
        "Action": "SERVICE_CREATE",
        "Resource": "svc-fquubrsk",
        "Details": "service nginx created with 1 replicas"
    },
    {
        "Timestamp": "2026-05-26T11:39:38.115845+03:00",
        "UserID": "system",
        "Action": "NODE_DEAD",
        "Resource": "worker-1",
        "Details": "node died with 1 containers"
    },
```

Figura 4.5. Audit log with recorded events

4.3. Timing and Performance Characteristics

Beyond confirming that each scenario behaves correctly, the timing of the most important operations was measured on the test machine by reading the timestamps in the master log. The goal is not a full performance benchmark but a check that the reaction times match the values configured in the code and that they are reasonable for a small cluster.

The fault tolerance module checks the nodes on a fixed five-second tick, so the detection times are observed in five-second steps rather than at the exact theoretical second. For example, a node that stops sending heartbeats is marked SUSPECT at the first check at or after 15 seconds, which in practice falls between 15 and 20 seconds. Table 4.1 summarizes the measured behavior.

Tabelul 4.1. Measured timing of the main operations

Operation	**Configured**	**Observed**
Heartbeat interval	5 s	5 s
Node discovery (first heartbeat to ALIVE)	$\leq$ 5 s	within one interval
SUSPECT detection	15 s	15–20 s
DEAD detection	30 s	30–35 s
Container recovery (DEAD to replacement RUNNING)	—	a few seconds after DEAD
Leader election after leader failure	—	1–4 s
Service deploy, image already present	—	under 1 s

The detection times follow the configured thresholds closely, with the small extra delay explained by the five-second check granularity. Container recovery starts immediately after a node is marked DEAD: the master reschedules the affected containers and the new ones reach the RUNNING state within a few seconds, the time being dominated by the worker creating and starting the container through Docker.

Leader election was measured by stopping the leader and reading the log of the master that took over. The new leader was elected in one to four seconds. This range comes from the HashiCorp Raft default timeouts (a one-second heartbeat timeout followed by a randomized election timeout), so the cluster keeps accepting write requests again within a few seconds of losing its leader.

Service deployment latency has two very different cases. When the Docker image is already present on the worker, the API responds in under a second, since the only work is creating and starting the container. The first deployment of a new image is instead dominated by the image pull from the registry, which depends on the image size and the network speed and is therefore not a property of the platform itself. For this reason only the image-present case is reported as a platform metric.

These measurements were taken on a single machine where the network latency between processes is negligible. On a real network the absolute values would grow with the round-trip time between nodes, but the relative behavior, detection in fixed steps and recovery within seconds, would stay the same.

Capitolul 5. Conclusions

This thesis presented the design and implementation of a distributed platform for running containerized services within a cluster. The platform was built from scratch in Go and addresses the core challenges of cluster management: automatic node discovery, resource allocation, load balancing, fault tolerance, security and auditing. The entire codebase is a single Go module with clear package structure, making it easy to read and extend.

5.1. Achieved Objectives

All objectives proposed in the introduction have been achieved:

- **Automatic node discovery**. Worker nodes are discovered automatically via UDP multicast heartbeats. No manual configuration or join tokens are needed. The master detects new nodes within 5 seconds of their first heartbeat.
- **Resource pools**. Nodes are grouped into pools based on CPU and RAM. Services are deployed to specific pools, ensuring they run on appropriate hardware.
- **Load balancing**. Two strategies are implemented (Least Connections and Weighted) behind a pluggable interface that allows custom strategies to be added without modifying existing code.
- **Fault tolerance**. The platform detects node failures within 30 seconds, automatically restarts containers on healthy nodes using the original service parameters and prevents duplicate containers through the KillContainers mechanism.
- **Security**. JWT authentication with refresh tokens and role-based access control (Admin, Writer, Reader) protect all API endpoints.
- **Audit**. All important actions are recorded in an append-only log queryable through the API.
- **High availability**. Raft consensus enables state replication across multiple master nodes with automatic leader election.
- **Docker integration**. Containers are managed through the Docker SDK, supporting image pulling, resource limits, environment variables, ports and custom commands.
- **REST API**. A complete HTTP API allows users to manage services, view nodes and query audit logs. The API is served by every master; write requests received by a follower are forwarded to the leader, so a leader change does not affect users.

5.2. Comparison with Similar Solutions

Compared to Kubernetes, Docker Swarm and Nomad (analyzed in Chapter 2), the proposed platform is simpler to understand and deploy. It implements the same fundamental concepts (heartbeats, consensus, scheduling, RBAC) but in a single Go module with minimal dependencies. This makes it suitable for educational purposes and smaller deployments where Kubernetes would be too complex.

The difference is clearest in how much is needed to run each system. A Kubernetes cluster needs a separate etcd database, several control-plane components and a network plugin and it is usually managed by a dedicated team. The proposed platform runs as two small binaries, the master and the worker, with no external services to install. A worker joins the cluster simply by starting and sending heartbeats and a service is deployed with a single HTTP request. This low setup cost is exactly what makes the platform a good fit for learning and for small clusters, where the time spent setting up Kubernetes would be larger than the problem being solved.

The main trade-off is that the platform lacks features expected in production systems: a durable and replicated audit log, service networking, horizontal scaling of masters and support for multiple

container runtimes. These are not gaps in the design but features left out on purpose to keep the system small and clear. Each of them is a natural next step rather than a change to the existing architecture, as described in the following section.

5.3. Future Work

Several improvements could extend the platform:

- **Durable audit**. Service and container state is already persisted through the Raft log, so a master rebuilds it on restart. The audit log, however, is kept only in memory and is not replicated; persisting and replicating it would be a useful next step.
- **Private image registries**. An ADMIN-only endpoint for registering the credentials of a private container registry, replicated through Raft and used by workers for authenticated image pulls (supported directly by the Docker SDK). This would let users deploy their own private images and would give the ADMIN role a capability that clearly separates it from WRITER. In production, the credentials should live in a secret manager rather than travel in plain text over the trusted network.
- **Dynamic master membership**. The set of master nodes is fixed at bootstrap through the `-raft-peers` flag, so adding a master requires restarting the cluster. A join endpoint on the leader, based on Raft's `AddVoter` operation, would add masters at runtime without downtime. The join must stay an explicit administrative action, because every new voter changes the quorum size of the cluster.
- **Other directions**. DNS-based service networking, auto-scaling of replicas, service health checks, a web dashboard and support for other container runtimes such as containerd or Podman.

5.4. Final Remarks

The introduction set out to manage containerized services across a cluster and to solve, in one system that is easy to understand, the main problems of node discovery, scheduling, fault recovery and access control. The platform solves all of them. Workers join the cluster on their own through heartbeats. The scheduler places a service only when there is room for all its replicas, so a deployment is never left half started. Failed nodes are detected and their containers are restarted on other nodes without any human action. Every request first passes through authentication, authorization and the audit log. The data that must survive a failure is copied to all master nodes with Raft and users always use the same REST API, no matter which master is the leader at that moment.

The work shows that the core mechanisms of a container orchestrator can be built from scratch and kept small enough to read and understand from start to finish. This was the main goal of the thesis. The aim was not to compete with Kubernetes on features, but to make the ideas behind it clear and easy to follow. The clear split between the master and the worker, the pluggable load balancer and the simple state machines for nodes and containers make the system easy to follow and easy to extend. This makes it a good base for the future work described above and a useful tool for learning how distributed systems work.

Bibliografie

[1] A. S. Tanenbaum and M. V. Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017, ISBN: 978-1-5430-5738-6.

[2] HashiCorp, "Consul documentation," https://developer.hashicorp.com/consul/docs, 2024, Ultima accesare: Mai 2026.

[3] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Addison-Wesley, 2003, ISBN: 978-0-13-141155-5.

[4] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, 2014, pp. 305–319.

[5] Docker Inc., "Docker overview," https://docs.docker.com/get-started/overview/, 2024, Ultima accesare: Mai 2026.

[6] The Kubernetes Authors, "Kubernetes documentation," https://kubernetes.io/docs/, 2024, Ultima accesare: Mai 2026.

[7] Docker Inc., "Swarm mode overview," https://docs.docker.com/engine/swarm/, 2024, Ultima accesare: Mai 2026.

[8] HashiCorp, "Nomad documentation," https://developer.hashicorp.com/nomad/docs, 2024, Ultima accesare: Mai 2026.

[9] Docker Inc., "Develop with docker engine sdks," https://docs.docker.com/engine/api/sdk/, 2026, Ultima accesare: Iunie 2026.

[10] HashiCorp, "hashicorp/raft: Golang implementation of the raft consensus protocol," https://github.com/hashicorp/raft, 2026, Ultima accesare: Iunie 2026.

[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, https://www.rfc-editor.org/rfc/rfc7519, 2015.

[12] The golang-jwt maintainers, "golang-jwt/jwt: Go implementation of json web tokens," https://github.com/golang-jwt/jwt, 2026, Ultima accesare: Iunie 2026.

Annexes

The following annexes contain the source code of the most important components of the platform: node discovery, failure detection and recovery, scheduling, load balancing and the Raft state machine. The data model is described in section 3.3 and the remaining packages (cluster state, REST API, security and Docker integration) are part of the complete source code submitted together with the thesis.

Annex 1. Heartbeat Receiver

```
1  package heartbeat
2
3  import (
4    "encoding/json"
5    "log"
6    "net"
7
8    "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
9    "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
10 )
11
12 type Receiver struct {
13   multicastAddr    string
14   state            *cluster.State
15   onLateHeartbeat  func(nodeID string)
16   onNodeDiscovered func(nodeID string)
17   conn             *net.UDPConn
18   stop             chan struct{}
19 }
20
21 func NewReceiver(multicastAddr string, state *cluster.State,
   onLateHeartbeat func(nodeID string), onNodeDiscovered func(nodeID
   string)) *Receiver {
22   return &Receiver{
23     multicastAddr:    multicastAddr,
24     state:            state,
25     onLateHeartbeat:  onLateHeartbeat,
26     onNodeDiscovered: onNodeDiscovered,
27     stop:             make(chan struct{}),
28   }
29 }
30
31 func (r *Receiver) Start() {
32   addr, err := net.ResolveUDPAddr("udp", r.multicastAddr)
33   if err != nil {
34     log.Fatalf("heartbeat receiver: resolve addr: %v", err)
35   }
36
37   conn, err := net.ListenMulticastUDP("udp", nil, addr)
38   if err != nil {
39     log.Fatalf("heartbeat receiver: listen: %v", err)
40   }
```

```
41      r.conn = conn
42      defer conn.Close()
43
44      conn.SetReadBuffer(8192)
45      log.Printf("heartbeat receiver: listening on %s", r.multicastAddr)
46
47      buf := make([]byte, 4096)
48      for {
49        n, _, err := conn.ReadFromUDP(buf)
50        if err != nil {
51          select {
52          case <-r.stop:
53            log.Println("heartbeat receiver: stopped")
54            return
55          default:
56            log.Printf("heartbeat receiver: read: %v", err)
57            continue
58          }
59        }
60        r.handleHeartbeat(buf[:n])
61      }
62    }
63
64    func (r *Receiver) Stop() {
65      close(r.stop)
66      if r.conn != nil {
67        r.conn.Close()
68      }
69    }
70
71    func (r *Receiver) handleHeartbeat(data []byte) {
72      var hb domain.Heartbeat
73      if err := json.Unmarshal(data, &hb); err != nil {
74        log.Printf("heartbeat receiver: unmarshal: %v", err)
75        return
76      }
77
78      node, exists := r.state.GetNode(hb.NodeID)
79      if !exists {
80        newNode := &domain.Node{
81          ID:       hb.NodeID,
82          Address:  hb.Address,
83          Status:   domain.NodeAlive,
84          TotalCPU: hb.TotalCPU,
85          UsedCPU:  hb.UsedCPU,
86          TotalRAM: hb.TotalRAM,
87          UsedRAM:  hb.UsedRAM,
88        }
89        r.state.AddNode(newNode)
90        if !r.state.AssignNodeToPool(hb.NodeID, newNode) {
91          log.Printf("heartbeat receiver: node %s does not match any pool
            and cannot receive containers", hb.NodeID)
92        }
```

```
93        log.Printf("heartbeat receiver: new node discovered: %s (%s)
          CPU=%.1f RAM=%dMB",
94          hb.NodeID, hb.Address, hb.TotalCPU, hb.TotalRAM)
95        if r.onNodeDiscovered != nil {
96          r.onNodeDiscovered(hb.NodeID)
97        }
98      } else if node.Status == domain.NodeDead && r.onLateHeartbeat != nil
        {
99        r.onLateHeartbeat(hb.NodeID)
100     }
101
102     r.state.UpdateNodeFromHeartbeat(hb)
103   }
```

Listing 1. internal/heartbeat/receiver.go – UDP multicast receiver with auto-join

Annex 2. Fault Tolerance

```
1   package fault
2
3   import (
4     "log"
5     "sync"
6     "time"
7
8     "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/agent"
9     "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
10    "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
11  )
12
13  type Tolerance struct {
14    state                *cluster.State
15    checkInterval        time.Duration
16    suspectAfter         time.Duration
17    deadAfter            time.Duration
18    isLeader             func() bool
19    applyContainerStatus func(containerID, status string)
20    onNodeDead           func(nodeID string, containers
      []*domain.Container)
21    mu                   sync.Mutex
22    recovering           map[string]bool
23    stop                 chan struct{}
24  }
25
26  func NewTolerance(state *cluster.State, checkInterval, suspectAfter,
    deadAfter time.Duration,
27    isLeader func() bool,
28    applyContainerStatus func(containerID, status string),
29    onNodeDead func(nodeID string, containers []*domain.Container),
30  ) *Tolerance {
31    return &Tolerance{
32      state:                state,
33      checkInterval:        checkInterval,
34      suspectAfter:         suspectAfter,
```

```
35        deadAfter:             deadAfter,
36        isLeader:              isLeader,
37        applyContainerStatus: applyContainerStatus,
38        onNodeDead:            onNodeDead,
39        recovering:            make(map[string]bool),
40        stop:                  make(chan struct{}),
41      }
42    }
43
44    func (t *Tolerance) Start() {
45      ticker := time.NewTicker(t.checkInterval)
46      defer ticker.Stop()
47
48      log.Printf("fault tolerance: started, check every %s, suspect after
        %s, dead after %s",
49        t.checkInterval, t.suspectAfter, t.deadAfter)
50
51      for {
52        select {
53        case <-ticker.C:
54          t.check()
55        case <-t.stop:
56          log.Println("fault tolerance: stopped")
57          return
58        }
59      }
60    }
61
62    func (t *Tolerance) Stop() {
63      close(t.stop)
64    }
65
66    func (t *Tolerance) check() {
67      nodes := t.state.GetAllNodes()
68      now := time.Now()
69
70      for _, node := range nodes {
71        if node.Status == domain.NodeDead  node.Status ==
          domain.NodeRemoved {
72          continue
73        }
74
75        since := now.Sub(node.LastHeartbeat)
76
77        switch {
78        case since >= t.deadAfter && node.Status != domain.NodeDead:
79          log.Printf("fault tolerance: node %s marked DEAD (no heartbeat
          for %s)", node.ID, since.Round(time.Second))
80          t.state.SetNodeStatus(node.ID, domain.NodeDead)
81
82          containers := t.activeContainersOnNode(node.ID)
83          if len(containers) > 0 && t.onNodeDead != nil {
84            t.onNodeDead(node.ID, containers)
85          }
```

```
86
87        case since >= t.suspectAfter && node.Status == domain.NodeAlive:
88          log.Printf("fault tolerance: node %s marked SUSPECT (no heartbeat
            for %s)", node.ID, since.Round(time.Second))
89          t.state.SetNodeStatus(node.ID, domain.NodeSuspect)
90        }
91      }
92    }
93
94    // activeContainersOnNode returns only containers that are (or are
      about to be)
95    // running on the node. Stopped or failed containers must not be
      recovered.
96    func (t *Tolerance) activeContainersOnNode(nodeID string)
      []*domain.Container {
97      var active []*domain.Container
98      for _, c := range t.state.GetContainersByNode(nodeID) {
99        switch c.Status {
100       case domain.ContainerRunning, domain.ContainerScheduled,
          domain.ContainerRescheduling:
101         active = append(active, c)
102       }
103     }
104     return active
105   }
106
107   // HandleLateHeartbeat handles a heartbeat from a node marked DEAD.
      Only the
108   // leader kills the duplicate containers and brings the node back to
      ALIVE,
109   // and only after the kill succeeds. Followers just mark the node
      ALIVE.
110   func (t *Tolerance) HandleLateHeartbeat(nodeID string) {
111     node, exists := t.state.GetNode(nodeID)
112     if !exists  node.Status != domain.NodeDead {
113       return
114     }
115
116     if t.isLeader != nil && !t.isLeader() {
117       t.state.SetNodeStatus(nodeID, domain.NodeAlive)
118       return
119     }
120
121     t.mu.Lock()
122     if t.recovering[nodeID] {
123       t.mu.Unlock()
124       return
125     }
126     t.recovering[nodeID] = true
127     t.mu.Unlock()
128
129     log.Printf("fault tolerance: late heartbeat from DEAD node %s,
      sending KillContainers", nodeID)
130
```

```
131     addr := node.Address
132     go func() {
133       defer func() {
134         t.mu.Lock()
135         delete(t.recovering, nodeID)
136         t.mu.Unlock()
137       }()
138
139       if _, err := agent.SendCommand(addr, agent.Command{Type:
          agent.CmdKillContainers}); err != nil {
140         log.Printf("fault tolerance: failed to send KillContainers to %s:
            %v", nodeID, err)
141         return
142       }
143
144       for _, c := range t.activeContainersOnNode(nodeID) {
145         t.setContainerStatus(c.ID, domain.ContainerStopped)
146       }
147
148       t.state.SetNodeStatus(nodeID, domain.NodeAlive)
149       log.Printf("fault tolerance: node %s back to ALIVE (containers
          killed, available for scheduling)", nodeID)
150     }()
151   }
152
153   func (t *Tolerance) setContainerStatus(containerID, status string) {
154     if t.applyContainerStatus != nil {
155       t.applyContainerStatus(containerID, status)
156       return
157     }
158     t.state.SetContainerStatus(containerID, status)
159   }
```

Listing 2. internal/fault/tolerance.go – Failure detection and recovery

Annex 3. Load Balancer

```
1   package loadbalancer
2
3   import
    "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
4
5   // LoadBalancer selects the best node from a list of candidates.
6   type LoadBalancer interface {
7     SelectNode(candidates []*domain.Node, service *domain.Service)
      *domain.Node
8   }
```

Listing 3. internal/loadbalancer/balancer.go – LoadBalancer interface

```
1   package loadbalancer
2
3   import
    "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
```

```
4
5   // LeastConnectionsLB picks the node with the fewest running
    containers.
6   type LeastConnectionsLB struct{}
7
8   func (lb *LeastConnectionsLB) SelectNode(candidates []*domain.Node,
    service *domain.Service) *domain.Node {
9     if len(candidates) == 0 {
10      return nil
11    }
12
13    best := candidates[0]
14    for _, n := range candidates[1:] {
15      if len(n.Containers) < len(best.Containers) {
16        best = n
17      }
18    }
19    return best
20  }
```

Listing 4. internal/loadbalancer/leastconn.go – Least Connections strategy

```
1   package loadbalancer
2
3   import
    "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
4
5   // WeightedLB picks the node with the highest combined free-CPU and
    free-RAM score.
6   type WeightedLB struct{}
7
8   func (lb *WeightedLB) SelectNode(candidates []*domain.Node, service
    *domain.Service) *domain.Node {
9     if len(candidates) == 0 {
10      return nil
11    }
12
13    best := candidates[0]
14    bestScore := score(best)
15    for _, n := range candidates[1:] {
16      s := score(n)
17      if s > bestScore {
18        best = n
19        bestScore = s
20      }
21    }
22    return best
23  }
24
25  // score normalizes free CPU and free RAM to a 0-1 range so they weigh
    equally.
26  func score(n *domain.Node) float64 {
27    var cpuScore, ramScore float64
28    if n.TotalCPU > 0 {
```

```
29      cpuScore = n.AvailableCPU() / n.TotalCPU
30    }
31    if n.TotalRAM > 0 {
32      ramScore = float64(n.AvailableRAM()) / float64(n.TotalRAM)
33    }
34    return cpuScore + ramScore
35  }
```

Listing 5. internal/loadbalancer/weighted.go – Weighted strategy

Annex 4. Scheduler

```
1   package scheduler
2
3   import (
4     "fmt"
5     "log"
6
7     "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
8     "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
9     "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/loadbalan
      cer"
10  )
11
12  type Scheduler struct {
13    state *cluster.State
14    lb    loadbalancer.LoadBalancer
15  }
16
17  func NewScheduler(state *cluster.State, lb loadbalancer.LoadBalancer)
    *Scheduler {
18    return &Scheduler{
19      state: state,
20      lb:    lb,
21    }
22  }
23
24  // Plan finds a node for every one of the n replicas, or fails without
25  // assigning anything (all-or-nothing). The resources and ports
    consumed by
26  // each replica are subtracted from the chosen node's free capacity
    before
27  // the next replica is placed, so a single request can never
    oversubscribe
28  // a node.
29  func (s *Scheduler) Plan(service *domain.Service, n int)
    ([]*domain.Node, error) {
30    poolNodes := s.state.GetPoolNodes(service.PoolID)
31    if len(poolNodes) == 0 {
32      return nil, fmt.Errorf("no nodes in pool %s", service.PoolID)
33    }
34
35    // Working copies of the alive nodes; the plan mutates these copies,
36    // never the real cluster state.
```

```
37      var working []*domain.Node
38      usedPorts := make(map[string]map[int]bool)
39      for _, pn := range poolNodes {
40        if !pn.IsAlive() {
41          continue
42        }
43        c := *pn
44        c.Containers = s.state.GetActiveContainersByNode(pn.ID)
45        working = append(working, &c)
46        ports := make(map[int]bool)
47        for _, ctr := range c.Containers {
48          if svc, ok := s.state.GetService(ctr.ServiceID); ok {
49            for _, p := range svc.Ports {
50              ports[p] = true
51            }
52          }
53        }
54        usedPorts[c.ID] = ports
55      }
56
57      plan := make([]*domain.Node, 0, n)
58      for i := 0; i < n; i++ {
59        var candidates []*domain.Node
60        for _, c := range working {
61          if c.AvailableCPU() >= service.RequiredCPU && c.AvailableRAM()
          >= service.RequiredRAM &&
62            !portConflict(usedPorts[c.ID], service.Ports) {
63            candidates = append(candidates, c)
64          }
65        }
66        if len(candidates) == 0 {
67          return nil, fmt.Errorf("can place only %d of %d replicas (need
          %.1f CPU, %d MB RAM each)",
68            i, n, service.RequiredCPU, service.RequiredRAM)
69        }
70        node := s.lb.SelectNode(candidates, service)
71        if node == nil {
72          return nil, fmt.Errorf("load balancer returned no node")
73        }
74        node.UsedCPU += service.RequiredCPU
75        node.UsedRAM += service.RequiredRAM
76        node.Containers = append(node.Containers,
          domain.Container{ServiceID: service.ID, Status:
          domain.ContainerScheduled})
77        for _, p := range service.Ports {
78          usedPorts[node.ID][p] = true
79        }
80        plan = append(plan, node)
81      }
82      return plan, nil
83    }
84
85    // Schedule finds the best node for a single replica of the service.
```

```go
func (s *Scheduler) Schedule(service *domain.Service) (*domain.Node, error) {
  plan, err := s.Plan(service, 1)
  if err != nil {
    return nil, err
  }
  node := plan[0]
  log.Printf("scheduler: selected node %s for service %s (%.1f CPU, %d MB RAM left after placement)",
    node.ID, service.Name, node.AvailableCPU(), node.AvailableRAM())
  return node, nil
}

// portConflict reports whether one of the requested ports is already used.
// Each port can be bound only once per machine.
func portConflict(used map[int]bool, ports []int) bool {
  for _, p := range ports {
    if used[p] {
      return true
    }
  }
  return false
}
```

Listing 6. internal/scheduler/scheduler.go – All-or-nothing replica planning

Annex 5. Raft FSM

```go
package cluster

import (
  "encoding/json"
  "fmt"
  "io"
  "log"

  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
  "github.com/hashicorp/raft"
)

// LogEntry is a command that modifies the cluster state via Raft.
type LogEntry struct {
  Type string          `json:"type"`
  Data json.RawMessage `json:"data"`
}

// Log entry types
const (
  LogAddService         = "AddService"
  LogRemoveService      = "RemoveService"
  LogAddContainer       = "AddContainer"
  LogSetContainerStatus = "SetContainerStatus"
)
```

```
26
27  // FSM implements raft.FSM for the cluster state.
28  type FSM struct {
29    state *State
30  }
31
32  func NewFSM(state *State) *FSM {
33    return &FSM{state: state}
34  }
35
36  // Apply is called by Raft when a log entry is committed.
37  func (f *FSM) Apply(l *raft.Log) any {
38    var entry LogEntry
39    if err := json.Unmarshal(l.Data, &entry); err != nil {
40      log.Printf("fsm: unmarshal error: %v", err)
41      return err
42    }
43
44    switch entry.Type {
45    case LogAddService:
46      var svc domain.Service
47      json.Unmarshal(entry.Data, &svc)
48      f.state.AddService(&svc)
49      log.Printf("fsm: apply AddService %s", svc.Name)
50
51    case LogRemoveService:
52      var args struct {
53        ID string `json:"id"`
54      }
55      json.Unmarshal(entry.Data, &args)
56      f.state.RemoveService(args.ID)
57
58    case LogAddContainer:
59      var c domain.Container
60      json.Unmarshal(entry.Data, &c)
61      f.state.AddContainer(&c)
62
63    case LogSetContainerStatus:
64      var args struct {
65        ID     string `json:"id"`
66        Status string `json:"status"`
67      }
68      json.Unmarshal(entry.Data, &args)
69      f.state.SetContainerStatus(args.ID, args.Status)
70
71    default:
72      log.Printf("fsm: unknown entry type: %s", entry.Type)
73    }
74
75    return nil
76  }
77
78  func (f *FSM) Snapshot() (raft.FSMSnapshot, error) {
79    return &fsmSnapshot{data: f.state.Snapshot()}, nil
```

```
80  }
81
82  func (f *FSM) Restore(rc io.ReadCloser) error {
83    defer rc.Close()
84    var snap Snapshot
85    if err := json.NewDecoder(rc).Decode(&snap); err != nil {
86      return fmt.Errorf("snapshot decode: %w", err)
87    }
88    f.state.Restore(snap)
89    log.Printf("fsm: restored state (%d nodes, %d services, %d
      containers)",
90      len(snap.Nodes), len(snap.Services), len(snap.Containers))
91    return nil
92  }
93
94  type fsmSnapshot struct {
95    data Snapshot
96  }
97
98  func (s *fsmSnapshot) Persist(sink raft.SnapshotSink) error {
99    defer sink.Close()
100   data, err := json.Marshal(s.data)
101   if err != nil {
102     sink.Cancel()
103     return fmt.Errorf("snapshot marshal: %w", err)
104   }
105   if _, err := sink.Write(data); err != nil {
106     sink.Cancel()
107     return fmt.Errorf("snapshot write: %w", err)
108   }
109   return nil
110 }
111
112 func (s *fsmSnapshot) Release() {}
```

Listing 7. internal/cluster/fsm.go – Raft Finite State Machine